

Drishti3D

A complete, plain-English guide to turning one drone flight into a real, measurable 3D model of the ground.

Problem owner National Technical Research Organisation (NTRO)

Category Software · Theme: Robotics and Drones

Official title Single-Pass Drone Video to Accurate 3D Model Generation System

Prepared for Naveen

Date 1 September 2026

This guide assumes you know nothing about 3D reconstruction, drones, or computer vision. Every technical word is explained the first time it appears, in ordinary language, with an everyday comparison. Nothing has been left out to make it shorter.

BEFORE YOU START

How to read this guide

You do not need to read this in order, but the first time through, you should. Each part uses words that were explained in the part before it.

This document explains a project called **Drishti3D**. It is a proposed piece of software, built for a national hackathon problem set by an Indian government technical agency. The short version of the goal is this: *a drone flies over an area once, recording video; the software turns that video into an accurate three-dimensional model of what the drone flew over, accurate enough that you can measure real distances and heights on it.*

That sentence contains about fifteen ideas that need unpacking. This guide unpacks all of them.

The four kinds of boxes you will see

EVERYDAY COMPARISON

A box like this takes a technical idea and compares it to something ordinary — a camera, a map, a tape measure. If a paragraph confuses you, look for the nearest box like this one.

THE WORD PEOPLE ACTUALLY USE

A box like this gives you the technical term for something that was just explained simply. You need these terms because they are what appear in the code, the documents, and the conversation with judges or engineers.

THE POINT

A box like this is a one-paragraph summary of the section. If you remember nothing else from a section, remember what is in this box.

CAREFUL HERE

A box like this flags something that is commonly misunderstood, or a claim that would be dishonest to make. This project has an unusual amount of these, and that is deliberate — honesty about accuracy is the core idea of the whole product.

A note on what this guide is based on

This guide is written from **the actual Drishti3D repository** — every Python module, every React screen, the API, the tests, the evaluation harness, the Docker setup and the bundled sample res-

ults — together with the project's original research brief. Where this guide says "the code does X", it is describing real, working code, and it names the file so you can go and look.

Two things follow from that. First, where the finished system differs from the original plan, this guide describes *what was built* and says so plainly. Second, where the bundled sample results tell a more complicated story than the summary claims, this guide shows you both numbers — Part 24 is entirely about that.

CONTENTS

What is in this guide

- 1 What this project actually is
The problem, who asked for it, and why it is hard
- 2 The whole system in one page
Video goes in, a measurable 3D model comes out
- 3 The vocabulary of 3D
Point clouds, meshes, textures, poses, keyframes
- 4 How a camera can measure the world
Parallax, triangulation, baseline, lenses, calibration
- 5 Turning a video into 3D points
Features, matching, Structure from Motion, bundle adjustment
- 6 The scale problem and the sensors that fix it
Why video alone cannot tell you size; GPS, IMU, RTK, PPK
- 7 Coordinates: where on Earth is this?
WGS84, ECEF, ENU — and the code that converts between them
- 8 Fitting the model onto the map
Umeyama, Sim(3), RANSAC, residuals, and levelling the ground
- 9 The honesty rule: provenance
The five trust classes and the single most important idea here
- 10 A guided tour of the repository
Every folder and file, and what each one is for
- 11 The pipeline, stage by stage
All fourteen stages of `pipeline.py`, in order
- 12 Inside the reconstruction engine
What `sfm.py` actually does, step by step
- 13 Removing things that move
Dynamic masking, and why it is switched off by default
- 14 Filling the holes with AI, honestly
The depth prior, and how it is kept out of measurements
- 15 Cleaning up and building a surface
Voxels, outliers, normals, Poisson meshing

- 16 **Measuring things**
Distance, height, area — and the safeguards around them
- 17 **The quality report**
Every number in it, and what it really tells you
- 18 **Exports: the file formats**
PLY, LAS, GLB, GeoJSON, CSV, JSON, HTML
- 19 **The backend**
FastAPI, the database, background jobs, live progress, security
- 20 **The API, endpoint by endpoint**
What an API is, and what each of these does
- 21 **The screens the user sees**
React, the five views, and the 3D viewer in the browser
- 22 **Proving it works: the evaluation harness**
ATE, ICP, completeness, and the datasets to run it on
- 23 **Tests, Docker, and running offline**
What is tested, and how it deploys air-gapped
- 24 **What the bundled sample runs actually show**
Two saved runs, two very different answers — and why
- 25 **Honest limits and how to demo this**
What to claim, what never to claim, what to say
- 26 **Glossary**
Every term in this guide, alphabetically
- 27 **The one-page cheat sheet**
Everything compressed, for the night before

PART 1

What this project actually is

A government agency wants software that watches a drone video and rebuilds the ground in 3D, accurately enough to measure. That is the entire brief. Everything else in this guide is about why that is hard and how it can be done.

1.1 Who is asking, and what did they ask for

The request comes from the **National Technical Research Organisation**, usually written as **NTRO**. It is an Indian government technical intelligence agency. They posted this as problem number **SIH26158** in the **Smart India Hackathon** — a national competition where students build working software for problems that real government departments have.

The official title of the problem is:

THE PROBLEM STATEMENT, WORD FOR WORD

"Single-Pass Drone Video to Accurate 3D Model Generation System."

Build an AI-enabled system that takes a single-pass UAV video, GPS coordinates and flight metadata, and produces a georeferenced and metrically accurate 3D representation — including terrain, structures, facades, rooftops, roads, infrastructure, vegetation, obstacles — as point clouds and textured meshes suitable for measurement, visualisation and analysis.

Let us take that apart word by word, because almost every word is doing real work.

"Single-pass"

The drone flies over the area **once**. It does not circle a building three times from different angles. It does not fly a careful grid pattern back and forth. It goes over, once, in one path, and that is all the footage you will ever get.

This matters enormously. Most professional 3D mapping from drones is done with a deliberate, repeated, overlapping flight plan — the drone flies a lawnmower pattern, taking still photos with 80% overlap, often flying the same area twice at different angles. That gives the software an enormous amount of information about every surface. A single pass gives you a fraction of that.

EVERYDAY COMPARISON

Imagine walking past a house once, on the footpath, filming with your phone. Now imagine walking all the way around the house twice, filming from every side. The first gives you a single-pass video. The second is what normal 3D mapping software expects. The first is much harder to work with — and it is what this project must handle.

"UAV video"

UAV means Unmanned Aerial Vehicle — a drone. **Video**, not photographs. This is another important detail. Video frames are usually lower quality than photos taken deliberately: they are more compressed, more likely to be motion-blurred, and the camera settings change automatically as the drone flies from a bright field into a shadow. But video is what you get from a drone that was simply flying and recording.

"GPS coordinates and flight metadata"

Alongside the video, the drone records a stream of numbers: where it was (latitude, longitude, altitude), which way it was pointing, how fast it was moving, and so on. This stream is called **telemetry**. It is what makes it possible to place the 3D model on a real map, and to know the real size of things — video by itself cannot tell you either. Part 6 explains exactly why.

"Georeferenced"

The finished 3D model is not floating in empty space with arbitrary numbers. Every point in it corresponds to a real place on Earth, expressible as a latitude and longitude. If you click a corner of a roof in the model, the software can tell you the actual coordinates of that corner on the planet.

"Metrically accurate"

"Metric" here means "in real units of length" — metres. The model is not just correctly *shaped*, it is correctly *sized*. If the software says a wall is 4.2 metres tall, and you go there with a tape measure, you should get roughly 4.2 metres. This is the difference between a pretty 3D picture and a measuring instrument, and it is the whole point of the project.

"Point clouds and textured meshes"

These are the two forms the output takes. Part 3 explains them properly. Very briefly: a **point cloud** is millions of individual coloured dots floating in 3D; a **textured mesh** is a solid surface with the photographs painted onto it.

1.2 What the system is for

NTRO is an intelligence organisation, so the uses skew towards situations where being wrong is expensive:

- **Intelligence and reconnaissance** — how tall is that structure, how wide is that gap, what is the layout of that compound.
- **Disaster response** — after an earthquake or a flood, fly over quickly and get a 3D map of what is standing, what has collapsed, and which roads are passable.
- **Infrastructure inspection** — bridges, towers, dams, power lines.
- **Planning and measurement** — volumes of material, areas of land, heights of obstacles.

In every one of these, someone will make a decision based on a number the software produced. That is why the project puts so much weight on being honest about which numbers are trustworthy.

1.3 The two hard truths at the centre of the project

The specification is unusually blunt about two limits that cannot be engineered away. They are not weaknesses of a particular method — they are consequences of physics and geometry. Understanding these two is the single most useful thing in this guide.

HARD TRUTH 1 — VIDEO ALONE CANNOT TELL YOU SIZE

A video of a scene can tell you the *shape* of that scene and the *shape* of the path the camera took, but not how big any of it is. A real building filmed from far away and a doll's house filmed from close up produce mathematically identical video. This is called **scale ambiguity**. The only way out is to bring in information from outside the video: GPS positions, altitude, a known object of known size, or a second camera at a known distance. Part 6 covers this in full.

HARD TRUTH 2 — YOU CANNOT MEASURE WHAT WAS NEVER FILMED

If the drone flew over a building once, the far side of that building never appeared in a single frame. No amount of processing recovers it, because the information was never captured. An AI can *guess* what the far side probably looks like, and that guess may look convincing — but it is a guess. Presenting it as a measurement is a lie, and in a disaster or intelligence context, a dangerous one.

Most 3D reconstruction products quietly paper over both of these. They fill in unseen surfaces so the model looks complete, and they do not tell you which parts were actually observed. Drishti3D's central design decision is to **refuse to do that** — to label every part of the model with where it came from and how much it should be trusted. Part 9 is entirely about this idea.

PART 2

The whole system in one page

Before the details, here is the shape of the whole thing. Read this once now, and again after you finish the guide — it will read very differently the second time.

2.1 The one-sentence version

The drone's video is chopped into individual pictures; the software works out where the camera was when it took each picture and what it was looking at; that gives thousands of 3D points; the GPS data is used to place those points correctly on Earth and at the correct real-world size; every point is labelled with how much it should be trusted; and the result is shown in a web browser where you can look at it, measure it, and export it.

2.2 The fourteen stages

The file `reconstruction/drishti_recon/pipeline.py` runs the whole job. It has exactly fourteen named stages, and those names appear in the progress bar in the browser as the job runs, so what you see on screen matches this list one-for-one.

```

Drone video (.mp4) + Telemetry (.csv / .json / .srt)
|
v
[ 1] ingestion      Open the video. Read its size, frame rate, duration,
                    | codec. Take a SHA-256 fingerprint of the file.
                    v
[ 2] telemetry      Parse the GPS file. Validate every row. Set up a local
                    | metric coordinate frame anchored on the flight.
                    v
[ 3] frames         Decode the video into images, skipping frames so that
                    | at most 240 are analysed, shrunk to 1280 px wide.
                    v
[ 4] quality        Score each frame: sharpness, brightness, clipping,
                    | motion, duplicates. Mark the bad ones with a reason.
                    v
[ 5] sync           Match each frame to the GPS reading from the same
                    | instant, interpolating between telemetry samples.
                    v
[ 6] keyframes      Choose the useful subset of frames – enough movement
                    | between them to triangulate, not so much they stop
                    | matching. Presets: fast / balanced / quality.
                    v
[ 7] masking        Optionally paint out moving objects. OFF by default,
                    | for a reason explained in Part 13.
                    v
[ 8] sfm            THE CORE. Work out every camera position and the 3D
                    | position of every matched feature. Output: a sparse
                    | cloud where each point came from >= 2 real sightings.
                    v
[ 9] densify        Optional. Use an AI depth model to fill in the holes.
                    | Everything it adds is labelled AI-assisted.
                    v
[10] georegistration Rotate, move and RESIZE the whole reconstruction so
                    | the camera path matches the GPS track. This is where
                    | real-world metres enter the system. Then level it.
                    v
[11] fusion         Thin out, remove stray points, compute surface
                    | directions, assign each point a trust class.
                    v
[12] mesh           Optional. Join the points into a solid surface.
                    |
                    v
[13] report         Compute every quality number. Compare against ground
                    | truth if a truth file is present.
                    v
[14] exports        Write PLY, LAS, GLB, GeoJSON, CSV, JSON and a
                    | printable HTML report, plus the browser viewer file.

```

2.3 Which engine actually does the reconstruction

This is worth getting straight immediately, because the original plan and the finished software differ here, and the difference is the most interesting engineering decision in the project.

The original brief called for **COLMAP** — a well-known, powerful, free reconstruction program — to be the main engine. COLMAP is excellent, but it is a separate program that has to be installed, it is awkward on Windows, and its best features want an NVIDIA graphics card.

What was actually built instead is a **complete reconstruction engine written from scratch** inside the project, in the file `sfm.py`, using only OpenCV — a standard image-processing library that installs with a single `pip install` and runs on any laptop's ordinary processor. Part 12 walks

through it. COLMAP is still supported, but as an *optional* alternative you can switch on if you have it.

So the honest picture of the engines is:

Engine	Status	What it is and when it runs
Built-in OpenCV SfM <code>sfm.py</code>	Default. AI-ways works.	A genuine feature-matching, triangulating, camera-registering reconstruction engine. Needs no extra installs, no graphics card, no downloaded AI models. This is the path whose output you are allowed to measure on.
COLMAP <code>colmap_adapter.py</code>	Optional	Used only if you install PyCOLMAP and ask for it. Higher accuracy on hard captures because it does full global bundle adjustment. The pipeline detects whether it is really installed and falls back if not.
Depth Anything V2 <code>depth_prior.py</code>	Optional, working	An AI model that guesses depth from a single image. Used to fill holes the classical engine cannot reach. Every point it adds is tagged AI-assisted and excluded from measurement. Part 14.
MASt3R-SLAM / VGGT <code>ai_adapter.py</code>	Stub only	The interface is fully defined and the availability check is honest, but the actual reconstruction call deliberately raises "not implemented". The code will never pretend these ran.

THE POINT

Fourteen stages take a video and a GPS file and produce a labelled, measurable, map-positioned point cloud. The reconstruction itself is done by a hand-written OpenCV engine that runs anywhere, with COLMAP and AI models as genuinely optional extras that the software is careful never to claim it used when it did not.

EVERYDAY COMPARISON

Think of it as a darkroom. The video is the undeveloped film. Stages 1–7 are sorting the negatives and throwing away the ruined ones. Stage 8 is the actual development — the picture appears. Stage 10 is putting a ruler and a map reference next to the print so you know how big everything is and where it was. Stages 11–14 are trimming, mounting, and writing the label that says exactly how the photograph was made.

PART 3

The vocabulary of 3D

Six words that appear constantly and mean specific things: point cloud, mesh, texture, model, pose, and frame. Get these right and half the jargon disappears.

3.1 A point cloud

A **point cloud** is exactly what it sounds like: a large collection of individual points floating in three-dimensional space. Each point is stored as three numbers — its position along three perpendicular directions, usually written **X**, **Y** and **Z**. Most points also carry a colour (red, green and blue values, taken from the photo the point was seen in).

A typical reconstruction from a drone video might contain anywhere from a few hundred thousand to tens of millions of points. Viewed from a distance, they look like a solid scene — like a photograph you can walk around inside. Viewed close up, you see they are separate dots with gaps between them.

EVERYDAY COMPARISON

A point cloud is like a swarm of glowing dust motes frozen in mid-air, each one sitting exactly on the surface of a real object, each one the colour of the surface at that spot. There is no "wall" — just a very dense layer of dots where the wall is.

Why it is the primary output. A point cloud is the most honest representation available, because each point corresponds to something the camera actually saw. Nothing is invented to fill in gaps. If the drone never saw a surface, there are simply no points there — a hole. That hole is *information*: it tells you honestly that this region was not observed. This is why the project treats the point cloud as the main deliverable and the mesh as an optional extra.

3.2 A mesh

A **mesh** is a continuous surface. Instead of loose dots, you have a skin: the points are joined into triangles, and the triangles together form solid-looking surfaces. Almost every 3D object in a video game or an animated film is a mesh.

Turning a point cloud into a mesh is called **meshing** or **surface reconstruction**. It is genuinely useful — meshes look better, are smaller files, and are what most other 3D software expects. But there is a catch that matters greatly here:

CAREFUL HERE

Meshing algorithms have to decide what to do about gaps. Most of them **fill the gaps in** — they stretch the surface across the hole to make a nice closed shape. That means a mesh can show you a smooth, confident-looking wall in a place where the drone never saw anything at all. The honest holes in the point cloud silently disappear. This is why Drishti3D keeps the point cloud as the authoritative output and treats the mesh as a visualisation aid, with the labels carried through so filled-in regions stay marked as inferred.

3.3 Texture

A bare mesh is grey and featureless — correct in shape, but it looks like a clay model. **Texturing** is the process of taking the original photographs and painting them onto the surface, so the model looks photographic: you can read a sign, see the tiles on a roof, tell brick from concrete.

Technically, this works by working out, for each triangle of the mesh, which photograph saw it best and from what angle, then projecting that piece of the photo onto that triangle. The result is a **textured mesh**, which is what most people picture when they hear "3D model".

3.4 A camera pose

A **pose** is the complete answer to "where was the camera and which way was it pointing?" at one instant. It has six numbers, called six **degrees of freedom**:

- **Position** — three numbers: how far east, how far north, how high.
- **Orientation** — three numbers: which compass direction it faces, how far it is tilted up or down, and how much it is rolled sideways. These three are commonly called **yaw**, **pitch** and **roll**.

EVERYDAY COMPARISON

Position tells you where you are standing. Orientation tells you where you are looking. You need both to know what someone can see. A camera at the same spot pointed at the sky and pointed at the ground has the same position but different poses.

The chain of poses for every keyframe, laid end to end, is the **camera trajectory** — the reconstructed flight path of the drone. This is one of the system's outputs in its own right, and it is drawn on a map in the interface.

3.5 A frame and a keyframe

A video is a rapid sequence of still images. Each still image is a **frame**. Video is usually 24, 30 or 60 frames per second, so a five-minute drone video at 30 frames per second contains 9,000 frames.

You do not want all 9,000. Consecutive frames one-thirtieth of a second apart are nearly identical, and processing both gives you almost no extra information for double the computation. Worse, having two nearly identical viewpoints is actively unhelpful for 3D — the reason is explained in Part 4.

So the software selects a subset: the **keyframes**. These are the frames that are sharp, well exposed, and meaningfully different from the one before. A few hundred well-chosen keyframes out of 9,000 frames is typical, and it produces a better result than using all of them.

3.6 Putting the words together

Term	In one line
Frame	One still picture out of the video.
Keyframe	A frame the software chose to actually use, because it is sharp and adds new information.
Pose	Where the camera was and which way it pointed for one keyframe: six numbers.
Trajectory	All the poses in order — the reconstructed flight path.
Point cloud	Millions of coloured 3D dots sitting on the surfaces the camera saw.
Mesh	Those dots joined into triangles to make a continuous surface.
Texture	The original photographs painted onto the mesh so it looks real.
Model	Loose word for the whole 3D result — cloud, mesh, or both.

THE POINT

The point cloud is the honest, primary output: dots where the camera actually saw a surface, and holes where it did not. The mesh is a prettier, more convenient derivative that risks inventing surfaces. Poses describe where the camera was; the sequence of poses is the trajectory; keyframes are the small useful subset of the video's frames.

PART 4

How a camera can measure the world

A single photograph flattens the world and throws depth away. Two photographs from different places bring it back. This part explains exactly how, and what a camera has to know about itself for the trick to work.

4.1 What a photograph loses

A camera takes the three-dimensional world and squashes it onto a flat rectangle of pixels. In doing so it throws away one piece of information for every point: **how far away it was**.

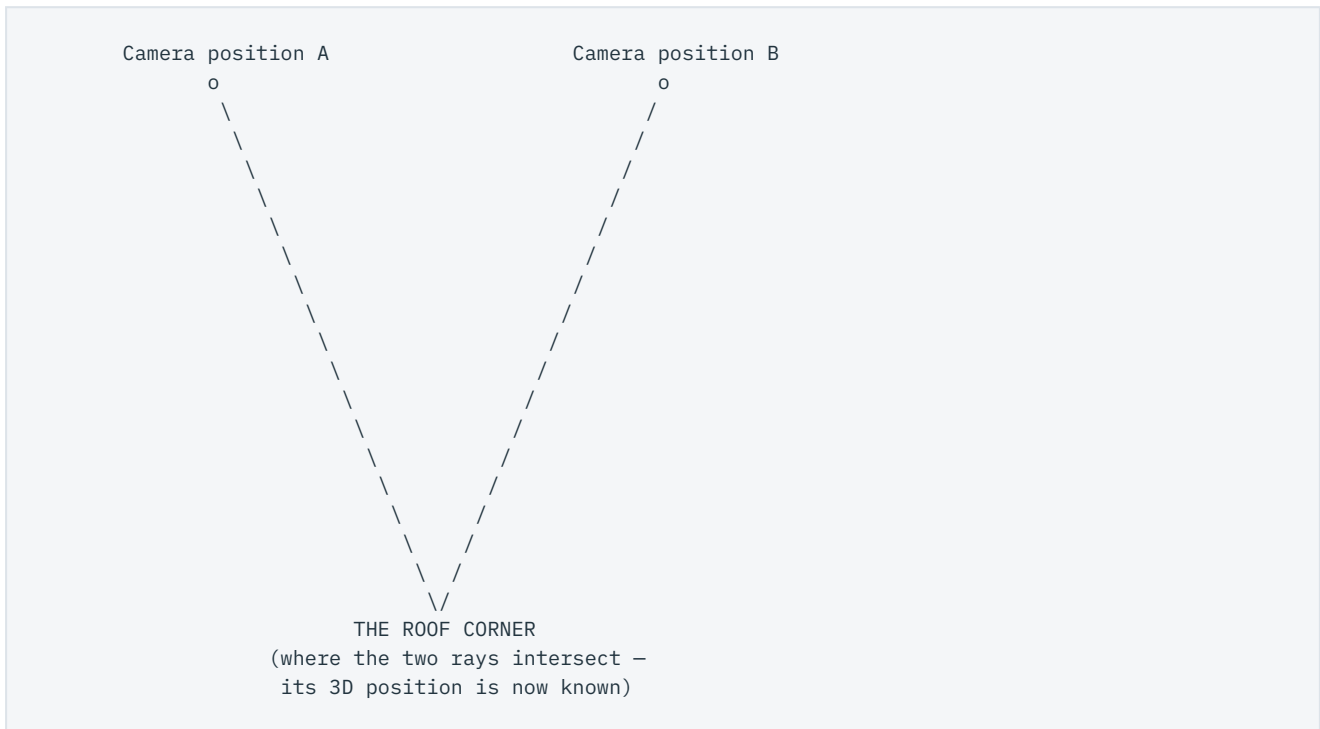
Consider one pixel in a photograph — say, a pixel showing the corner of a roof. That pixel tells you a direction: "something was along this line, out from the camera." It does not tell you how far along that line. The roof corner could be 10 metres away or 100 metres away; it would land on the same pixel either way, just at a different apparent size.

THE WORD PEOPLE ACTUALLY USE

That line out from the camera through a pixel is called a **ray**. A single photograph gives you one ray per pixel and no information about position along the ray. Recovering the missing distance is called **depth estimation**, and doing it for a whole image gives you a **depth map** — a picture where each pixel's value is "how far away this was" instead of a colour.

4.2 What two photographs give you back

Now take a second photograph of the same roof corner from a different position. You now have two rays — one from each camera position — and both pass through the roof corner. Two lines in 3D space that both pass through the same point intersect at that point. So you can compute where the corner is: it is where the two rays cross.



This is the entire foundation of 3D reconstruction from ordinary cameras. Do it for one point and you get one 3D point. Do it for a million points across hundreds of images and you get a point cloud.

THE WORD PEOPLE ACTUALLY USE

Computing a 3D position from two or more rays is called **triangulation** — the same word surveyors have used for centuries, and the same idea. The apparent shift of an object against its background when you move the camera is called **parallax**.

EVERYDAY COMPARISON

Hold a finger up at arm's length and close one eye, then the other. Your finger jumps sideways against the background. Now hold it close to your nose and repeat — it jumps much further. Your brain uses exactly that difference to judge distance. Two eyes a few centimetres apart give you depth perception over a few metres; a drone camera at two positions fifty metres apart gives you depth perception over hundreds of metres. Same principle, bigger baseline.

4.3 Baseline: why the camera must move enough

The distance between the two camera positions is called the **baseline**. It matters enormously, and it explains one of the most important design decisions in this system.

If the two camera positions are very close together, the two rays are nearly parallel. Two nearly parallel lines cross at a very shallow angle, and the exact point where they cross becomes extremely sensitive to tiny errors. A one-pixel error in finding the roof corner in either image moves the computed 3D position by a huge amount. The depth estimate is unreliable.

If the two camera positions are well separated, the rays cross at a healthy angle and the intersection is sharply defined. A one-pixel error barely moves the answer.

THE WORD PEOPLE ACTUALLY USE

The angle at which the two rays meet is the **triangulation angle** (also called the parallax angle or intersection angle). A large angle means a well-determined point; a small angle means a poorly-determined one. Drishti3D records this angle for every point and uses it as one input to that point's confidence score — a point triangulated at 2° is far less trustworthy than one triangulated at 30° .

CAREFUL HERE — THE TRADE-OFF

You might conclude "always use a huge baseline". But a large baseline has the opposite problem: the two images look so different that the software struggles to recognise that a feature in one is the same feature in the other. Surfaces get hidden, lighting changes, perspective distorts. So there is a sweet spot — enough separation for good geometry, not so much that matching fails. This is precisely what **keyframe selection** (Part 10, stage 4) is trying to hit: frames far enough apart to triangulate well, close enough to still match.

Note what this means for the single-pass constraint. A drone flying in a straight line gives you baselines along the direction of flight, and those grow as you compare frames further apart. But you never get a baseline *across* the flight line, and you never get to see the far side of anything. The geometry is decent along the path and absent everywhere else.

4.4 What the camera must know about itself: intrinsics

The ray-intersection trick assumes you know, for each pixel, exactly which direction it corresponds to. That is not automatic — it depends on the specific camera and lens. The numbers describing this are called the camera's **intrinsic parameters**, or **intrinsics**.

There are four main ones, and you will see them written exactly like this in the code and configuration:

Symbol	Name	What it means in plain terms
<code>fx</code>	Focal length, horizontal	How "zoomed in" the camera is, measured across the image, in pixels. A large value means a narrow, telephoto view; a small value means a wide-angle view. This single number controls the relationship between how far away something is and how big it appears.
<code>fy</code>	Focal length, vertical	The same thing measured up and down. On almost all modern cameras <code>fx</code> and <code>fy</code> are nearly equal, because pixels are square. If they differ substantially, the pixels are not square.
<code>cx</code>	Principal point, horizontal	Where the exact optical centre of the lens falls on the image, measured in pixels from the left edge. In theory this is the middle of the image; in practice the sensor is never mounted perfectly, so it is a bit off.
<code>cy</code>	Principal point, vertical	The same, measured from the top edge.

Lens distortion

Real lenses bend light imperfectly. Straight lines in the world come out slightly curved in the image — most visibly on wide-angle and action cameras, where the effect is called **barrel distortion** or, in its extreme form, the fisheye look. Drone cameras are usually wide-angle, so this is significant.

Distortion is described by a handful of extra numbers, the **distortion coefficients**, conventionally named `k1`, `k2`, `k3` (radial distortion — the curving-outward effect) and `p1`, `p2` (tangential distortion — a slight skew from the lens not being perfectly parallel to the sensor).

CAREFUL HERE

If you ignore distortion, every ray is pointing slightly the wrong way, and the error grows towards the edges of the image. The reconstruction will still produce a result — but a subtly bent one, where a flat football field comes out as a shallow bowl. This is one of the classic silent failures in this field, and it is why the specification insists on either using supplied calibration or estimating it and *reporting the uncertainty*.

Calibration

Determining these numbers for a specific camera is called **camera calibration**. The standard method is almost comically low-tech: you print a chessboard pattern, tape it to something flat, take twenty or thirty photos of it from different angles with the camera you care about, and let software work backwards. Because the software knows a chessboard has perfectly straight lines and perfectly equal squares, any deviation in the photos must be the lens's doing — and from that it solves for `fx`, `fy`, `cx`, `cy` and the distortion coefficients.

Drishti3D handles three situations:

- **Calibration supplied** — the user enters the known intrinsics. Best case; the geometry is properly grounded.
- **Intrinsics in the video metadata** — some drones record focal length in the file. Usable, approximate.
- **Nothing supplied** — the software estimates the intrinsics as part of the reconstruction itself. This works surprisingly well but adds uncertainty, and the system is required to say so rather than pretend otherwise.

THE POINT

Depth comes from seeing the same thing from two different places and intersecting the rays. The quality of that depth depends on the **baseline** — too small and the answer is unstable, too large and matching fails. For any of it to work, the software must know the camera's **intrinsics** (f_x , f_y , c_x , c_y) and its **lens distortion**, either measured beforehand by calibration or estimated during processing with the uncertainty declared.

PART 5

Turning a video into 3D points

Part 4 assumed you already knew where the two cameras were. In reality you know neither the camera positions nor the 3D points — you have to solve for both at once, from nothing but the pictures. This part explains how that is possible.

5.1 The chicken-and-egg problem

Here is the awkward situation. To triangulate a 3D point, you need to know where the cameras were. To work out where the cameras were, it helps enormously to know where some 3D points are. You have neither.

The way out is that the two unknowns constrain each other. Given enough matched points across enough images, there is essentially only one arrangement of cameras and points that explains all the observations consistently. The software finds that arrangement by starting with a rough guess and refining it repeatedly.

5.2 Step one: features

You cannot match every pixel — a pixel in the middle of a blank grey road looks like every other pixel on that road. So the software first finds **features**: small, visually distinctive patches that can be recognised again from a different angle and at a different brightness. Corners, junctions, blobs, spots of texture. The corner of a window. A crack in the tarmac. A distinctive bush.

For each feature, the software computes a **descriptor** — a compact numerical summary of what the patch looks like, designed so the same physical patch photographed from a different angle produces a similar summary, while a different patch produces a very different one.

THE WORD PEOPLE ACTUALLY USE

The individual points are **keypoints** or **feature points**. The numerical summaries are **descriptors**. Classic algorithms for producing them are **SIFT**, **ORB** and **AKAZE**; modern learned alternatives include **SuperPoint** for detection and **SuperGlue** or **LightGlue** for matching. COLMAP, the verified engine in this project, uses SIFT by default.

5.3 Step two: matching

Feature matching means, for each feature in image A, finding the feature in image B with the most similar descriptor — the claim being "these two are the same physical thing seen twice."

Many of these claims are wrong. Two different windows on the same building look nearly identical. Repeated patterns — tiles, fences, crop rows, waves — are a nightmare. So matching always produces a mixture of correct matches and confident nonsense.

The nonsense is removed geometrically. If a set of matches is correct, all of them are consistent with *one* plausible relative movement of the camera between the two shots. Wrong matches are not consistent with that movement. So the software looks for the single camera movement that the largest number of matches agree with, and discards the rest as errors. The technique for doing this is called **RANSAC**, which is explained properly in Part 8 because it comes up again there.

THE WORD PEOPLE ACTUALLY USE

A match that survives this geometric check is an **inlier**; one that is thrown out is an **outlier**. The mathematical relationship that valid matches between two images must obey is called the **epipolar constraint**, encoded in a small matrix called the **fundamental matrix** (or the **essential matrix** when the intrinsics are known).

Matching strategy: why "sequential"

Comparing every image to every other image is the thorough approach, but for 500 keyframes that is about 125,000 comparisons — expensive. Drone video has a helpful property: consecutive frames are consecutive in space too. So the specification calls for **sequential matching** — compare each keyframe only to its near neighbours in time — plus a limited number of **loop closure candidates**.

EVERYDAY COMPARISON — LOOP CLOSURE

Loop closure is what happens when you are walking through an unfamiliar building, taking mental notes of every turn, and you suddenly recognise a corridor you have already been down. Instantly you can correct your accumulated sense of where you are. The software does the same: if it recognises that keyframe 400 is looking at the same place as keyframe 12, it can snap the whole trajectory into consistency and eliminate accumulated drift.

5.4 Step three: Structure from Motion

Now the actual solving. **Structure from Motion** — nearly always written **SfM** — is the classical technique for recovering both the 3D structure of a scene and the motion of the camera from a set of overlapping images. It works incrementally:

1. Pick two images with plenty of good matches and a healthy baseline. Compute the camera movement between them. This defines an arbitrary starting coordinate system.
2. Triangulate the matched features between those two images. You now have a small cloud of 3D points.
3. Take a new image. Some of the features in it match points you have already placed in 3D. Since you know where those points are in space and where they appear in the new image, you can solve for where the new camera must have been. (This step has its own name: **PnP**, for Perspective-n-Point.)

4. With the new camera placed, triangulate the features it shares with earlier images that were not yet in 3D. The cloud grows.
5. Refine everything (see bundle adjustment, below).
6. Repeat from step 3 with the next image, until all images are placed or fail to place.

THE WORD PEOPLE ACTUALLY USE

An image successfully placed into the reconstruction is **registered**. The percentage of keyframes that get registered is one of the headline quality numbers in this project — if only 40% of your keyframes registered, something went badly wrong (usually blur, poor texture, or too-fast motion).

5.5 Step four: bundle adjustment

Every step above introduces small errors, and they accumulate. **Bundle adjustment** is the correction: a single enormous optimisation that adjusts all the camera poses and all the 3D point positions simultaneously, to make everything as consistent as possible.

The measure of consistency is the **reprojection error**, and it is worth understanding precisely because it is the most quoted quality number in the whole field.

REPROJECTION ERROR, PRECISELY

Take one 3D point in your reconstruction. Take one camera that saw it. Using the camera's computed pose and intrinsics, calculate mathematically where that 3D point *should* appear in that camera's image — which pixel it lands on. Now compare that to where the feature was *actually detected* in the image. The distance between the two, in pixels, is the reprojection error for that point in that image.

A good reconstruction has a median reprojection error under about one pixel. Above two or three pixels, something is wrong: bad intrinsics, bad matches, or motion blur.

Bundle adjustment nudges every camera pose and every point position, over and over, to make the total reprojection error across all points in all images as small as possible. The name comes from the "bundle" of light rays converging on each camera, which the process adjusts.

EVERYDAY COMPARISON

It is like tightening the spokes on a bicycle wheel. Tightening one spoke alone pulls the rim out of true; you have to adjust all of them together, a little at a time, until the whole wheel is round. Bundle adjustment tightens thousands of "spokes" — every observation of every point by every camera — until the whole structure is as true as it can be.

5.6 SLAM, and how it differs from SfM

SLAM stands for **Simultaneous Localisation and Mapping**: working out where you are (localisation) while building a map of your surroundings (mapping) at the same time, with neither known in advance. It is the same underlying problem as SfM, but with a different priority.

	Structure from Motion (SfM)	SLAM
Designed for	Processing a batch of images afterwards	Running live, as the images arrive
Priority	Accuracy — take the time needed	Speed — keep up with the camera
Image order	Does not need to know it	Relies on it heavily
Typical use	Survey mapping, photogrammetry, film	Robots, self-driving cars, AR headsets, drones navigating
In this project	The built-in <code>sfm.py</code> engine — and COLMAP as an option	Not used. MAST3R-SLAM was planned as a fast preview path but its adapter is a stub.

The word **dense** attached to either ("dense SLAM", "dense reconstruction") means it produces depth for essentially every pixel rather than only at sparse feature points. Dense results look like real surfaces; sparse results look like a scattering of dots. Dense is much more computationally expensive and usually needs a GPU.

5.7 Sparse versus dense, and why it matters here

Classical SfM produces a **sparse point cloud** — one point per successfully triangulated feature, perhaps tens of thousands of points. It is accurate but skeletal; you can make out the shape of buildings but not read detail.

Dense reconstruction is a follow-up stage that goes back to the images with the now-known camera poses and computes depth for every pixel, producing millions of points. In COLMAP this stage is called **MVS** — Multi-View Stereo — and it generally requires an NVIDIA GPU with CUDA.

CAREFUL HERE — WHAT THIS MEANS FOR DRISHTI3D

Drishti3D produces a **sparse** cloud. Dense multi-view stereo is not implemented, because it would require the GPU and CUDA-enabled COLMAP the project deliberately refuses to depend on. So a typical result here is thousands of points, not millions.

Two things fill the gap. The AI depth prior (Part 14) adds density, but every point it adds is labelled AI-assisted and cannot be measured. And the viewer's "splat" mode draws points as soft overlapping blobs so a sparse cloud still reads as a surface. Neither is dense reconstruction, and the software does not pretend otherwise — the "degrade honestly rather than fail or fake" pattern runs through the whole design.

THE POINT

Find distinctive patches (features), match them across images, throw out the wrong matches geometrically, then grow a reconstruction one image at a time (SfM), constantly refining all cameras and all points together to minimise reprojection error (bundle adjustment). SLAM is the same maths arranged for real-time use. Sparse gives you tens of thousands of accurate points; dense gives you millions but needs a GPU.

PART 6

The scale problem and the sensors that fix it

This is the part of the project that most people find genuinely surprising. A perfect video, perfectly processed, cannot tell you how big anything is. Here is why, and what fixes it.

6.1 Why video alone cannot give you size

Everything in Part 5 — features, matching, triangulation, bundle adjustment — recovers *shape*. It tells you that the camera moved in a certain path and that the scene has a certain form. What it cannot tell you is the **size** of either.

The reason is a simple symmetry. Suppose you have a correct reconstruction. Now multiply every camera position and every 3D point by two. The scene is twice as big and the camera moved twice as far. Ask what the camera would see: every ray from every camera to every point is pointing in exactly the same direction as before, because doubling both the distance to a thing and the size of that thing leaves its appearance unchanged. So every image is **pixel for pixel identical**.

If two different-sized worlds produce identical images, no amount of cleverness applied to those images can distinguish them. The information is not hidden — it was never captured.

EVERYDAY COMPARISON

This is exactly why film-makers can fake a giant monster with a model, and why a photograph of a sandcastle can look like a real fortress. Without a person, a car, or a ruler in shot, a photograph carries no size information at all. Your brain normally solves this by recognising familiar objects — "that's a door, so the wall is about three metres." The software has no such assumptions, so it needs a real measurement from outside.

THE WORD PEOPLE ACTUALLY USE

This is called **scale ambiguity**, and a reconstruction that has shape but not size is said to be **up to scale** or in an **arbitrary scale**. Getting real units into it is **metric scale recovery**. In the code, a run that never achieved this is labelled `scale_source: "relative"` and the report warns that it is "shape only; not georeferenced, not metric".

6.2 What can break the ambiguity

You need one real length measured by something that is not the camera. Any of these will do:

Source	How it fixes scale, and how good it is
GPS positions	You know where the drone actually was, in metres, at many moments. Compare that to the reconstructed camera path, and the ratio between the two gives you the scale factor. This is what Drishti3D uses. Accuracy is limited by the GPS itself — typically a few metres for ordinary consumer GPS.
RTK or PPK GPS	A far more accurate form of satellite positioning (below). Centimetre-level, so the recovered scale is much better.
A known object	You tell the software "this thing in the scene is exactly 2.00 m long." One such measurement scales everything. Surveyors call these ground control points or GCPs.
Two cameras a known distance apart	A stereo rig. The fixed separation is the ruler. Not applicable here — a single-camera drone is the premise of the problem.
A laser or radar rangefinder	Directly measures a distance. Some survey drones have one; the problem statement does not assume it.

6.3 GPS, and what it actually gives you

GPS (more properly **GNSS**, since Europe, Russia, China and India all run their own satellite systems too) works by timing radio signals from satellites and solving for the receiver's position on Earth.

An ordinary GPS receiver — in a phone or a consumer drone — is accurate to roughly **2 to 10 metres horizontally**, and typically two to three times worse *vertically*. Altitude is the weak axis of satellite positioning, for geometric reasons: the satellites are all above you, so the up-down direction is poorly constrained by their arrangement.

CAREFUL HERE — THIS IS A HARD CEILING ON THE PRODUCT

If the GPS positions the whole model is anchored to are individually wrong by 3 metres, then **the model's absolute position on Earth is wrong by something like 3 metres**. No processing fixes this; the error is in the input. This is why the code and its report repeat the phrase "absolute accuracy is **GPS-limited**". Note carefully that this does not mean measurements *within* the model are off by 3 metres — averaging many GPS readings along the flight gives a much better scale estimate than any single reading, so relative measurements can be far more accurate than the absolute position. But you cannot promise centimetre placement on Earth from metre-accurate GPS.

RTK and PPK

These are two ways of getting satellite positioning down to centimetres.

- **RTK** — Real-Time Kinematic. A second GPS receiver sits still on the ground at a known location, a **base station**. Because it is not moving and its true position is known, it can meas-

ure exactly how wrong the satellite signals are at that moment, and radio the correction to the drone *while it flies*. The drone applies the correction live and records centimetre-accurate positions.

- **PPK** — Post-Processed Kinematic. Identical idea, except the correction is not radioed live. Both receivers log raw data, and the two logs are combined on a computer after landing. Slightly more robust, since there is no radio link to lose.

Drishti3D looks for a telemetry column called `rtk_status` (or `rtk`, or `fix_type`) and treats values like `RTK`, `FIXED` or `4` as meaning the reading is RTK-quality. If any row says so, the report labels the scale source as `rtk` instead of `gps` — so a reader of the report can tell at a glance which accuracy regime the numbers belong to.

6.4 The other telemetry, and what each field is for

The file `telemetry.py` accepts three formats — **CSV** (a spreadsheet-style table), **JSON** (a structured text format), and **SRT** (the subtitle file that DJI drones record alongside video, with the flight data written into the subtitle text). Four fields are required; the rest are optional but helpful.

Field	Required	What it is
<code>timestamp</code>	Yes	When this reading was taken, in seconds. Used to line the reading up with the right video frame. If missing, the code uses the row number instead.
<code>latitude</code>	Yes	How far north or south, in degrees. Must be between -90 and $+90$ or the row is rejected.
<code>longitude</code>	Yes	How far east or west, in degrees. Must be between -180 and $+180$.
<code>altitude</code>	Yes	Height in metres. Must be a finite number.
<code>roll</code>	No	How much the aircraft is tipped left or right, in degrees.
<code>pitch</code>	No	How much it is tipped nose-up or nose-down. (<code>gimbal_pitch</code> is accepted as the same thing — the gimbal is the motorised mount that keeps the camera steady.)
<code>yaw</code>	No	Which compass direction it faces. Also accepted as <code>heading</code> or <code>compass_heading</code> .
<code>velocity</code>	No	Speed over the ground, in metres per second.
<code>gps_accuracy</code>	No	The receiver's own estimate of how wrong it might be, in metres. Also accepted as <code>hdop</code> , <code>accuracy</code> or <code>hacc</code> . Genuinely used: readings that claim to be more accurate are given more influence when the model is fitted to the GPS track.
<code>rtk_status</code>	No	Whether this was an RTK-quality fix, as described above.
<code>fx fy cx cy</code>	No	The camera intrinsics from Part 4. If your telemetry carries them, the software will use them and skip the guess.

THE POINT

The telemetry file is not a nice-to-have — without it the reconstruction has no size and no place on Earth. Four fields are essential: time, latitude, longitude, altitude. The parser is deliberately forgiving about what the columns are called, but it never silently drops a bad row: every rejected row appears in the run's warnings with its row number and the reason.

6.5 What the IMU is, and why it barely features here

An **IMU** — Inertial Measurement Unit — is the sensor package that measures acceleration and rotation. It is the chip that knows your phone has been turned sideways. On a drone it reports roll, pitch and yaw many times a second, far faster than GPS updates.

In serious drone reconstruction systems, IMU data is fused with the camera and GPS to keep track of orientation through moments when the camera is confused. Drishti3D reads the orientation fields if present and interpolates yaw sensibly, but it does not do full inertial fusion. The reconstruction gets its orientation from the images themselves. This is a reasonable simplification: orientation from images is generally very good, and it is *position and scale* that images cannot supply.

THE WORD PEOPLE ACTUALLY USE

Interpolating yaw needs care because angles wrap around. Going from 350° to 10° is a 20° turn through north, not a 340° turn the other way. The function `_angle_interp` in `sync.py` handles this by always taking the shorter way round, and there is a test asserting that halfway between 350° and 10° is 0°.

PART 7

Coordinates: where on Earth is this?

Three different ways of writing down a position, each used for a different job. Mixing them up is the classic catastrophic bug in geographic software, and the code has a comment warning about it.

7.1 The problem with latitude and longitude

Latitude and longitude are angles, not distances. Latitude is the angle north or south of the equator; longitude the angle east or west of Greenwich. They describe a position on a curved surface, which is correct and useful, but it makes them **terrible for arithmetic**.

Here is the trap. One degree of latitude is always about 111 kilometres. But one degree of longitude is about 111 km at the equator, roughly 96 km in Delhi, about 78 km in London, and *zero* at the North Pole — because the lines of longitude converge as you go north. So the distance corresponding to a degree depends on where you are.

CAREFUL HERE

You cannot subtract two latitude/longitude pairs and treat the answer as a distance. You cannot average them naively over a large area. You cannot compute an area by treating them as *x* and *y*. Doing so gives an answer that looks plausible and is wrong by an amount that varies with location. This is why the code file `geo.py` opens with the instruction: *"Never mix geographic degrees with metric coordinates."*

7.2 The three coordinate systems used here

WGS84 — latitude, longitude, altitude

WGS84 is the World Geodetic System of 1984 — the standard mathematical model of the Earth's shape, and the one GPS uses. It describes the planet as an **ellipsoid**: a sphere slightly squashed at the poles, with an equatorial radius of 6,378,137 metres and a flattening of about 1 part in 298. Those two constants appear literally in the code, as `_A` and `_F`.

When your drone reports a position, it is a WGS84 latitude, longitude and altitude. This is the system used for **storing and exchanging** positions — every map, every GPS, every GeoJSON export.

CAREFUL HERE — TWO KINDS OF ALTITUDE

WGS84 altitude is height above the mathematical **ellipsoid**, not height above sea level. Those differ, in places by tens of metres, because the actual sea-level surface (the **geoid**) is lumpy where the Earth's gravity is stronger or weaker. Drishti3D asks for ellipsoidal altitude and does not attempt a geoid correction. For measuring heights *within* a model this does not matter at all, because a constant offset cancels out. For claiming an absolute elevation above sea level, it would matter.

ECEF — Earth-Centred, Earth-Fixed

ECEF puts the origin at the centre of the Earth and measures three ordinary distances in metres: **X** out through the point where the equator crosses the Greenwich meridian, **Y** out through the equator 90° east of that, and **Z** straight up through the North Pole. "Earth-fixed" means the axes rotate with the planet, so a fixed building keeps fixed coordinates.

ECEF has the great virtue of being ordinary metres you can do arithmetic with, and it works anywhere on Earth without exception. Its drawback is that the numbers are enormous and unintuitive — a point in India has coordinates around 1,300,000 / 5,900,000 / 1,400,000 metres, and a one-metre feature is a change in the seventh digit. Drishti3D uses ECEF purely as a **stepping stone**: it is the mathematically clean bridge between angles and local metres.

ENU — East, North, Up (the one that matters)

ENU is the system all the actual work happens in. You choose one point on Earth as the **origin**, and then measure everything as three plain distances from it:

- **E** — metres east of the origin (negative means west)
- **N** — metres north of the origin (negative means south)
- **U** — metres up from the origin (negative means down)

EVERYDAY COMPARISON

ENU is what you do naturally when giving directions locally: "the shed is 30 metres east and 12 metres north of the back door, and the roof is 3 metres up." You have picked an origin — the back door — and everything else is plain metres from it. Perfect for a small area; useless for describing somewhere in another country.

ENU is called a **local tangent plane** frame, because it treats the Earth as flat — as the flat plane that just touches the globe at your origin. Over the size of a drone survey this is an excellent approximation; the error from ignoring curvature is millimetres over a kilometre. Over hundreds of kilometres it would fall apart, but nothing here is that big.

In the code, ENU is represented by the class `ENUFrame`, which stores its origin as `lat0`, `lon0`, `alt0` and offers two methods: `geodetic_to_enu` to convert GPS readings into local metres, and

`enu_to_geodetic` to convert local metres back into GPS coordinates. The origin is created by `ENUFrame.from_points`, which simply averages all the telemetry positions — so the origin sits in the middle of the flight, keeping every coordinate small and well-behaved.

THE POINT

WGS84 (angles) is for **storing and exchanging** positions. ECEF (huge metres from the Earth's centre) is the **bridge**. ENU (plain metres from a local origin) is where all the **real work** happens: the reconstruction lives in it, measurements are computed in it, and it is what "metrically accurate" means in practice. Because the origin is always kept, any ENU point can be turned back into a latitude and longitude on demand.

7.3 How the conversion is actually done

The code uses **pyproj**, the standard Python interface to PROJ, the cartographic projection library that most of the world's geographic software depends on. Two coordinate reference systems are named by their EPSG codes: `EPSG:4979` is WGS84 with a height, and `EPSG:4978` is WGS84 as ECEF. (EPSG codes are a universal numbering system for coordinate systems, so software can agree on exactly which one is meant.)

Notably, the file also contains a complete **hand-written fallback** using only NumPy, for the case where `pyproj` fails to import. Going from angles to ECEF is direct trigonometry; coming back is not, because the ellipsoid makes the equations non-invertible, so the fallback uses **Bowring's method** — an iterative technique that guesses the latitude and refines it six times, converging to well below a millimetre. That fallback existing is a small sign of the project's general character: no single dependency is allowed to be the thing that stops it running.

The step from ECEF to ENU is a rotation. Given the origin's latitude and longitude, there is a fixed 3×3 matrix whose three rows are the directions "east", "north" and "up" at that spot, written in ECEF terms. Multiply an ECEF offset by that matrix and you get east, north and up. The function `_enu_rotation` builds exactly that matrix.

PART 8

Fitting the model onto the map

You have a reconstruction with the right shape but arbitrary size, position and orientation. You have a GPS track in real metres. This part is the mathematics that snaps one onto the other — and it is where "metrically accurate" is actually delivered.

8.1 What has to change, and what must not

The reconstruction needs to be moved, turned and resized until its camera path lies on top of the GPS track. Exactly four things are allowed to change:

- **Position** — slide the whole thing (3 numbers).
- **Orientation** — turn the whole thing (3 numbers).
- **Scale** — uniformly grow or shrink it (1 number).

That is seven numbers total, and critically, **scaling must be uniform** — the same factor in all three directions. If you allowed different stretch factors per axis you could squash the model to fit the GPS more closely, and you would be destroying the very shape the reconstruction worked so hard to recover. A building would come out taller than it is, to make a residual smaller. The whole point is that the shape is trustworthy and only its size is unknown.

THE WORD PEOPLE ACTUALLY USE

A transformation that rotates, translates and uniformly scales is a **similarity transform**. In three dimensions the group of such transformations is written **Sim(3)** — "similarity in 3 dimensions", seven degrees of freedom. You will see it constantly in this field. The related term **SE(3)** means rotation and translation *without* scaling, six degrees of freedom. In the code, `Sim3` is a small class holding `scale`, a rotation matrix `R` and a translation `t`, applying them as `y = scale × (R · x) + t`.

8.2 Finding the best fit: the Umeyama method

Given a set of reconstructed camera positions and the corresponding GPS positions, there is a known, exact procedure for the single best-fitting similarity transform: **Umeyama's algorithm**, published in 1991. It is not a search or an iteration — it computes the answer directly:

1. Find the centre of each set of points and shift both to the origin. This removes the translation from the problem.
2. Compute the **covariance** between the two centred sets — a 3×3 matrix summarising how they correspond directionally.

3. Apply the **SVD** (singular value decomposition) to that matrix. This is a standard linear-algebra operation that splits a matrix into a rotation, a scaling along axes, and another rotation. Out of it falls the optimal rotation.
4. Compare the overall spread of the two point sets to get the scale factor.
5. Recover the translation from the two centres and the now-known rotation and scale.

One subtlety in the code deserves a mention because it is a classic trap. The SVD can hand back a "rotation" that is actually a **reflection** — a mirror image, which would fit the points just as well numerically but would produce a mirrored world where east and west are swapped. The standard guard is to check the determinant and, if it is negative, flip the last axis. The three lines doing this in `umeyama_sim3` are what stop the reconstruction coming out inside-out.

8.3 The problem with best fits: outliers

Umeyama gives the transform that minimises the total squared error. That is exactly the right answer *if* all the correspondences are basically correct. It is a terrible answer if a few are badly wrong.

And some will be. GPS occasionally produces a wild reading — a satellite drops out, a signal reflects off a building. A camera can be mis-registered by the reconstruction. Because the method squares the errors, one point that is 50 metres off contributes as much as a hundred points that are 5 metres off, and it drags the entire fit towards itself.

EVERYDAY COMPARISON

Imagine averaging the heights of thirty people to estimate a typical height, and one entry was mistyped as 300 cm. The average is now wrong for everybody. What you want is a method that notices the 300 cm figure does not belong with the others and simply ignores it.

8.4 RANSAC: how the code ignores bad data

RANSAC stands for **RANdom SAMple Consensus**. It is one of the most useful ideas in computer vision, and it appears three separate times in this project — in feature matching, in camera registration, and here. The idea is disarmingly simple:

1. Pick the smallest number of points that can define a fit — here, three camera-to-GPS pairs.
2. Compute the transform from just those three.
3. Apply it to *all* the points and count how many land close to where they should — within 3 metres, in this code. Those are the **inliers**; the rest are **outliers**.
4. Remember this attempt if it produced more inliers than any previous attempt.
5. Go back to step 1 with a different random three. Repeat up to 500 times.
6. Finally, take the best attempt's inlier set and recompute the transform properly using *all* of them.

The logic is that a good sample of three will agree with all the other good points, producing a large consensus. A sample containing a bad point produces a transform that agrees with almost nothing. So the largest consensus almost certainly comes from good data.

Two details in the implementation are worth knowing:

- **Collinearity check.** If the three sampled camera positions lie in a straight line, they do not define a unique orientation — the model could be spun about that line freely. The code computes a cross product and skips any such sample. This matters enormously for this project, because *a single-pass drone flight is nearly a straight line*. Part 8.6 comes back to this.
- **Graceful degradation.** If RANSAC never finds a decent consensus, the code does not fail. It falls back to using every point in a plain least-squares fit, which is the best that can be done with the data available.

8.5 Weighting by GPS accuracy

If the telemetry includes a `gps_accuracy` column, the pipeline builds weights as `1 / accuracy` and passes them in. A reading that claims to be accurate to 0.4 m gets more than six times the influence of one that admits to 2.5 m. This is the statistically correct thing to do: trust the measurements that deserve trust.

8.6 What comes out, and how to read it

The result is a `Sim3Result` carrying the transform plus its diagnostics:

Field	What it means
<code>scale</code>	The factor the reconstruction was multiplied by to reach real metres. Its absolute value is meaningless (it depends on the arbitrary starting scale), but a run where it changes wildly between attempts is a warning sign.
<code>n_inliers / n_total</code>	How many cameras agreed with the fit. If a large fraction were rejected, either the GPS or the reconstruction has a problem.
<code>alignment_rmse_horizontal_m</code>	Typical east–north disagreement between the fitted camera path and the GPS track, in metres.
<code>alignment_rmse_vertical_m</code>	The same for the up direction.

THE WORD PEOPLE ACTUALLY USE

RMSE means Root Mean Square Error. Take every error, square it, average the squares, take the square root. It is a "typical error" that deliberately punishes large errors more than small ones. It is the standard way accuracy is quoted in this field.

CAREFUL HERE — THE MOST IMPORTANT CAVEAT IN THE WHOLE REPORT

The alignment RMSE is **not** an accuracy figure, and the code says so in a note that is printed in the report and shown on screen: *"Alignment residual is NOT independent accuracy; it measures consistency between reconstructed camera centres and GPS."*

Here is why this matters. That number tells you how well two things agreed with each other. But both could be wrong in the same way. If the GPS has a systematic 3-metre bias, the reconstruction will be fitted to that bias, agree with it beautifully, and report a tiny residual while sitting 3 metres from where it should be. A low residual means "internally consistent", not "correct". Real accuracy requires comparing to something independent — surveyed ground points, or a ground-truth dataset. Part 22 is about exactly that.

8.7 Levelling the ground

There is one more correction, and the reason for it is a nice illustration of how the single-pass constraint bites in unexpected places.

A mapping drone typically flies at nearly constant altitude. That means the GPS altitude values barely change over the whole flight — so when the alignment fits the reconstruction to the GPS, the *vertical* direction is very weakly constrained. There is almost no information telling it which way is up. The result can be a reconstruction that is correct in shape but subtly tilted, so the ground appears as a gentle slope in the viewer, and worse, a "height" measurement measures partly along the ground.

The function `level_rotation` in `geo.py` fixes this using an assumption that is safe in aerial imagery: **the biggest flat surface in the scene is the ground, and the ground is horizontal**. It uses Open3D's `segment_plane` — itself a RANSAC procedure — to find the largest plane in the cloud, then computes the rotation that would bring that plane's normal direction to point straight up, and applies it to the points and the cameras.

It is guarded carefully, which is the interesting part:

- At least 200 points, or it does nothing — too little data to trust.
- The plane must contain at least 35% of all points, or it is not the ground.
- If the correction needed is under 1°, skip it — already level, do not fiddle.
- If the correction needed is over 80°, refuse — that plane is a wall or a roof, not the ground, and applying it would wreck the model.
- If Open3D is not installed, return nothing and carry on.

When it does act, it appends a warning to the run so the user knows the model was rotated: *"applied gravity/level correction so the ground plane is horizontal in the viewer."* Nothing happens silently.

PART 9

The honesty rule: provenance

If you take one idea away from this guide, take this one. It is what makes Drishti3D different from other 3D reconstruction software, it is the answer to the hardest question a judge can ask, and it is a whole file of code — `provenance.py` — that could not be simpler.

9.1 The problem it solves

A finished 3D model looks uniformly confident. Every surface is drawn the same way. Looking at it, you cannot tell which parts were photographed from six angles in perfect light, which were glimpsed once at a bad angle, which were filled in by an algorithm's best guess, and which are simply not there at all.

For a pretty visualisation, that does not matter. For someone deciding whether a vehicle fits through a gap, or whether a damaged building is safe to enter, it matters enormously — and the model gives them no way to tell the difference.

EVERYDAY COMPARISON

Imagine a map where roads that were surveyed, roads that were sketched from memory, and roads that someone assumed must exist are all drawn with the same confident black line. The map is not wrong exactly — but it is dangerous, because it hides how much of itself you should doubt. Now imagine the same map with surveyed roads solid, remembered roads dashed, and assumed roads dotted. Same information, radically more useful.

9.2 The five classes

Every single point in a Drishti3D cloud carries a number from 0 to 4 recording where it came from:

Class	Measurable?	What it means
 <code>OBSERVED_HIGH_CONFIDENCE</code> code 0	Yes	Seen properly by several cameras, from good angles, and the geometry checks out. This is measurable, defensible geometry.
 <code>OBSERVED_LOW_CONFIDENCE</code> code 1	Flagged	Genuinely seen — not invented — but weakly: few observations, a shallow triangulation angle, or a larger reprojection error. Usable with caution; measuring on it raises a warning.
 <code>AI_ASSISTED</code> code 2	No (opt-in)	Produced by a neural network's estimate of depth, not by triangulating real observations. Plausible, useful to look at, excluded from measurement unless you explicitly tick a box.
 <code>DYNAMIC_EXCLUDED</code> code 3	No	Identified as belonging to something that was moving — a car, a person. Deliberately kept out of the static model.
 <code>UNOBSERVED</code> code 4	No	No camera ever saw this. The honest answer is a hole in the model, and the hole is information.

9.3 How the class is decided

For observed geometry it comes from a confidence score between 0 and 1, computed in `sfm.py` as a weighted blend of three things that each independently indicate how well a point was pinned down:

```
confidence = 0.40 x (how many cameras saw it)
            + 0.35 x (how small its reprojection error is)
            + 0.25 x (how wide its triangulation angle is)

where:
  observations : 2 sightings -> 0.0, 6+ sightings -> 1.0
  reprojection : 4 px error  -> 0.0, 0 px error   -> 1.0
  angle        : 0 degrees   -> 0.0, 10+ degrees  -> 1.0
```

Every one of those three inputs was explained earlier: observation count and reprojection error in Part 5.5, triangulation angle in Part 4.3. This is why those sections mattered — they are literally the ingredients of the trust score.

Then the split is a single threshold, in `provenance.py` : **0.6 or above is high confidence, below 0.6 is low**. That is the entire function. There is something admirable about the most philosophically important part of the system being four lines of code.

9.4 How the rule is enforced everywhere

A label is worthless if nothing acts on it. This one is enforced at four separate layers, which is what makes it a real guarantee rather than a claim:

- **In the measurement code.** `measure.py` filters out AI-assisted points before it even looks for the nearest point to what you clicked. You cannot accidentally measure to inferred geometry — the point is not in the set being searched.
- **In the fusion step.** `add_inferred_layer` appends AI-derived points as a distinct block with confidence set to exactly zero and provenance set to AI-assisted. The comment is explicit: inferred geometry "is never merged into observed geometry."
- **In the viewer.** The 3D display has a "provenance" colour mode that paints the cloud by trust class rather than by real colour, plus a checkbox per class to hide or show it. You can literally switch off everything except high-confidence geometry and see what is left.
- **In the exports.** The PLY file format is extended with a per-point `confidence` value and a `provenance` code, so the labelling survives being opened in other software. The LAS export stores confidence in the intensity field.

9.5 The measurement warnings

When you do take a measurement, the code inspects the class of every point involved and attaches warnings:

- Touched anything AI-assisted (only possible if you opted in) → *"measurement touches AI-assisted geometry"*
- Touched anything low-confidence → *"measurement touches low-confidence geometry"*
- Neither → the measurement is noted as *"high-confidence"*

These warnings are stored in the database with the measurement itself, appear in the interface, and are included when measurements are exported. A number that came from shaky geometry carries that fact with it forever, rather than being laundered into a clean-looking figure in a report.

THE POINT

Every point knows where it came from, on a five-level scale from "measured properly" to "never seen". Measurements silently refuse to use inferred geometry, the viewer can colour and filter by trust, and the labels survive export. The system's honesty is not a promise in a document — it is enforced in four places in the code, and there are tests that check it.

PART 10

A guided tour of the repository

What is where, and what each piece is for. If you are ever lost in the project, come back to this page.

10.1 The top level

The repository contains two quite different things: a folder of research and planning documents, and the actual working software. The software all lives under `drishti3d/`.

```

Drishti3D/
├── claude_research/          research dossiers (planning, not code)
├── codex_research/          more research and architecture notes
├── DRISHTI3D_PROJECT_DRAFT.md
├── SIH26158_..._prompt.md    the original brief this was built from
├── drishti3d/               <-- THE ACTUAL SOFTWARE
│   ├── reconstruction/      the computer-vision engine (pure Python library)
│   ├── backend/            the web server and database
│   ├── frontend/          the browser interface
│   ├── eval/              the accuracy-proving harness
│   ├── tests/             automated tests
│   ├── docs/              user documentation
│   ├── sample_data/        a generated demo dataset + two saved runs
│   ├── odm_data_bellus.../  ~120 real aerial photos, for testing on real data
│   ├── docker-compose.yml  one-command deployment
│   ├── .env.example        all the configuration settings, documented
│   └── README.md

```

THE WORD PEOPLE ACTUALLY USE

This layout is a **monorepo** — one repository holding several distinct components that are developed together. The alternative would be separate repositories for the library, the server and the interface. A monorepo is simpler when the pieces change together, which they do here.

10.2 The reconstruction library — the heart of it

Everything scientific lives in `reconstruction/drishti_recon/`. It is a proper installable Python package with no dependency on the web server at all — you could use it from a script, which is exactly what the tests and the evaluation harness do.

File	Size	What it does
<code>pipeline.py</code>	21 KB	The conductor. Runs all fourteen stages in order, reports progress, writes every output file. The largest file, but it contains almost no clever logic — it wires the others together.
<code>sfm.py</code>	12 KB	The reconstruction engine. Features, matching, triangulation, camera registration. Part 12 covers it in full.
<code>geo.py</code>	9 KB	All coordinate conversion and the Sim(3) alignment. Parts 7 and 8.
<code>exports.py</code>	8.8 KB	Writes PLY, LAS, GLB, GeoJSON, CSV, JSON and the HTML report. Part 18.
<code>synth.py</code>	8.7 KB	Generates the synthetic test dataset — renders a known 3D scene from a moving camera, so accuracy can be checked against exact truth.
<code>telemetry.py</code>	7.3 KB	Parses and validates CSV / JSON / SRT telemetry. Part 6.4.
<code>depth_prior.py</code>	7.2 KB	The AI depth model that fills holes. Part 14.
<code>masking.py</code>	6.3 KB	Finding and masking out moving objects. Part 13.
<code>quality.py</code>	5.4 KB	Computes every statistic and assembles the quality report. Part 17.
<code>fusion.py</code>	4.7 KB	Cleans the cloud and assigns provenance classes. Part 15.
<code>measure.py</code>	4.3 KB	Distance, height, area, point measurements. Part 16.
<code>ai_adapter.py</code>	3.6 KB	The interface for MAST3R-SLAM and VGGT. Honest stubs.
<code>keyframes.py</code>	3.1 KB	Choosing which frames to actually use. Part 11, stage 6.
<code>sync.py</code>	2.9 KB	Matching frames to GPS readings in time. Part 11, stage 5.
<code>frame_quality.py</code>	2.8 KB	Scoring frames for blur, exposure, motion. Part 11, stage 4.
<code>ingestion.py</code>	2.7 KB	Opening the video, reading its properties, extracting frames.
<code>mesh.py</code>	2.0 KB	Turning the cloud into a surface. Part 15.
<code>provenance.py</code>		The five trust classes. Part 9. The smallest and most important file.

File	Size	What it does
	1.9 KB	
<code>colmap_adapter.py</code>	7.9 KB	The optional COLMAP engine.

10.3 The rest

Folder	Contents
<code>backend/app/</code>	Nine small files: <code>main.py</code> (the app), <code>config.py</code> (settings), <code>db.py</code> and <code>models.py</code> (database), <code>schemas.py</code> (data validation), <code>storage.py</code> (safe file uploads), <code>jobs.py</code> (background processing), and four <code>routers/</code> files defining the API. Part 19.
<code>frontend/src/</code>	<code>api.ts</code> (the typed connection to the backend), five view files under <code>views/</code> , plus <code>PointCloudViewer.tsx</code> (the 3D display) and <code>TrajectoryMap.tsx</code> (the flight-path plot). Part 21.
<code>eval/</code>	Five files that run the pipeline against datasets with known truth and score it. Part 22.
<code>tests/</code>	Seven test files covering coordinates, telemetry, quality scoring, measurements, API security, the evaluation code, and a full end-to-end run. Part 23.
<code>sample_data/</code>	A generator script, a synthetic flight (video + telemetry + exact ground truth), and two <i>saved complete runs</i> — which turn out to be very revealing. Part 24.
<code>odm_data_bellus-master/</code>	About 120 real geotagged aerial photographs from the OpenDroneMap sample collection, plus a ground-control-point list. Real test data.

PART 11

The pipeline, stage by stage

This is the guided walk through `pipeline.py`. Each stage gets its own section, with what goes in, what happens, and what comes out.

11.0 How progress is reported

Before the stages, one mechanism. The pipeline defines a fixed map from stage names to slices of the progress bar:

ingestion	0%	->	5%	densify	72%	->	78%
telemetry	5%	->	8%	georegistration	78%	->	84%
frames	8%	->	20%	fusion	84%	->	90%
quality	20%	->	28%	mesh	90%	->	94%
sync	28%	->	32%	report	94%	->	97%
keyframes	32%	->	40%	exports	97%	->	100%
masking	40%	->	48%				
sfm	48%	->	72%	<-- the big one, a quarter of the bar			

When a stage reports it is 50% done, the pipeline maps that onto the stage's slice and reports the overall figure. The effect is a progress bar that moves steadily and never jumps backwards. Every stage is also timed, and those timings appear in the final report — so you can see exactly where the minutes went.

11.1 Stage 1 — Ingestion

In: a video file. **Out:** its properties and a fingerprint.

The code opens the video with OpenCV and reads its width, height, frames per second, total frame count and codec. Two details show care:

- Some video containers lie about their frame count, or report zero. If that happens, the code re-opens the file and counts the frames by stepping through them, capped at 100,000 so a corrupt file cannot hang the program.
- The **SHA-256** of the file is computed and stored. This is a cryptographic fingerprint: a 64-character string derived from the file's contents such that any change, even one byte, produces a completely different string. It goes into the report, so anyone reading the results can verify which exact file produced them. This is the difference between a result and a piece of evidence.

THE WORD PEOPLE ACTUALLY USE

A **codec** is the compression scheme the video is stored in — H.264 is the common one. The code reads it via a **FOURCC**, a four-character identifier packed into a number, and unpacks it back into letters. The project deliberately avoids requiring a system FFmpeg install by shipping a bundled one through the `imageio-ffmpeg` package, so there is one less thing to install.

11.2 Stage 2 — Telemetry

In: a CSV, JSON or SRT file (or nothing at all). **Out:** validated GPS samples and a local ENU frame.

Format is detected from the file extension. Every row is checked: latitude in range, longitude in range, altitude a finite number. Bad rows are skipped *and recorded* — `"row 17: invalid/missing lat/lon/alt, skipped"` — never dropped in silence. Column names are matched against a table of aliases, so `lat`, `lon`, `heading`, `hdop` and so on all work without the user having to rename anything.

Then the samples are sorted by time, RTK status is checked, and an `ENUFrame` is created anchored at the average of all the positions.

THE NO-GPS PATH

Telemetry is optional. If none is supplied, or fewer than two valid rows survive, the pipeline switches to **relative-scale mode**: it reconstructs the shape and warns explicitly that the result is *"shape only; not georeferenced, not metric"*. Everything downstream still works, but nothing can be measured in real units. This is the right behaviour — degrade and declare, rather than fail or fake.

11.3 Stage 3 — Frames

In: the video. **Out:** a list of images in memory.

Two limits are applied, and they are the main reason the software runs on an ordinary laptop:

- **At most 240 frames** are analysed. The code computes a stride — if the video has 9,000 frames, it takes every 38th one. A single-pass video is highly redundant; 240 well-spread frames carry nearly all its geometric information.
- **At most 1280 pixels wide.** A 4K frame is shrunk by a factor of three. This is not just for speed — feature detection on a downscaled image is often *more* reliable, because shrinking averages away sensor noise and compression artefacts.

CAREFUL HERE

If the frames are resized, the camera intrinsics must be resized too. A focal length of 2000 pixels at 4K becomes about 667 pixels at 1280 wide. The function `_resolve_intrinsics` does this scaling. Forgetting it is a classic bug that produces a reconstruction which is subtly, consistently wrong.

Where the intrinsics come from

Immediately after decoding, the camera parameters are resolved in strict priority order:

1. Explicit `fx, fy, cx, cy` entered by the user — used directly, scaled to the processing size.
2. A `focal_length` found in the telemetry — used, with the image centre assumed for `cx, cy`.
3. Nothing available — the code guesses **focal = $0.9 \times$ the larger image dimension**, a reasonable rule of thumb for typical cameras, and adds this warning to the run: *"camera intrinsics not provided; estimated focal = $0.9 * \max(w,h)$. Provide fx,fy,cx,cy for higher accuracy."*

That warning is not decoration. Both bundled sample runs carry it, and it is a real contributor to their error figures.

11.4 Stage 4 — Frame quality

In: the frames. **Out:** a score card per frame, with accept/reject and reasons.

Blur, measured by the Laplacian

The **Laplacian** is an image operation that responds to rapid change in brightness — edges. Applied to a sharp photo it produces strong responses at every edge; applied to a blurry photo, where everything is smoothly smeared, it produces weak responses everywhere. So the **variance** of the Laplacian (how spread out its values are) is a good single-number sharpness score. High variance means sharp; low means blurry. The threshold here is 40.

EVERYDAY COMPARISON

It is the same intuition as judging a photo by whether the edges look "crisp". The Laplacian just measures crispness numerically instead of by eye.

Exposure

Three numbers are computed from the greyscale image: mean brightness, the fraction of pixels below 16 (essentially black — **clipped dark**), and the fraction above 240 (essentially white — **clipped bright**). Clipping matters because clipped pixels contain *no information at all*: everything darker than black records as black, so texture in that region is gone permanently and no feature can ever be found there.

Motion and duplicates

Each frame is shrunk to 64×64 and normalised, then compared to the previous one. Large difference means fast motion. Near-zero difference means the frame is a near-duplicate — the drone was hovering, or the video stalled — and it is flagged, because a duplicate adds computation without adding any new viewpoint.

The thresholds are all in one small configuration object (`QualityThresholds`), and a frame's rejection is recorded as a list of specific reasons: `blurry`, `underexposed`, `overexposed`, `clipped_dark`, `clipped_bright`, `duplicate`. Those reasons are shown in the interface when you hover over the frame timeline.

11.5 Stage 5 — Synchronisation

In: frame timestamps and GPS samples. **Out:** a GPS position for every frame.

The GPS updates maybe 5 or 10 times a second; the video runs at 30. So most frames fall *between* two GPS readings and their position must be interpolated.

Crucially, the interpolation happens **in ENU metres, not in latitude and longitude** — which is exactly the discipline Part 7 warned about. Positions blend linearly. Yaw blends with the wrap-around handling described earlier. GPS accuracy blends linearly too.

Each frame also gets a **synchronisation residual** — how far in time the nearest real GPS sample was — and a confidence derived from it, which decays as the residual grows relative to the overall span of the telemetry. A configurable time offset is supported for the common case where the video clock and the telemetry clock do not start together.

11.6 Stage 6 — Keyframe selection

In: all analysed frames plus their quality scores. **Out:** the subset that will actually be reconstructed.

Part 4.3 explained the trade-off: enough camera movement between frames to triangulate well, not so much that matching fails. This stage is that trade-off in code. It walks through the accepted frames and takes a new keyframe when *any* of three conditions is met:

1. **Enough visual change** — measured by **optical flow**, an algorithm (Farnebäck's) that estimates how far each pixel appears to have moved between two images. The average movement must exceed the preset's threshold.
2. **Too long since the last one** — a forced keyframe after a maximum gap, so the sequence never breaks.
3. **The drone moved a metre** — measured from the GPS, which catches real movement even when the visual change is subtle.

Three presets tune this:

Preset	Min. flow	Min gap	Max gap	Cap	Use when
fast	18 px	3	25	40	Quick preview; fewer, more separated keyframes.
balanced	10 px	2	18	80	The default.
quality	6 px	1	12	160	Best result; four times as many keyframes as fast, and much slower.

Two safety nets bracket the selection, and the code comments explain why both exist:

- **A floor of 30 keyframes.** If the rules produced fewer, the code spreads extra keyframes evenly through the sequence. The comment records what went wrong before this was added: *"Under-sampling here starves SfM — it was the cause of near-empty urban reconstructions."* A slow or hovering flight produces little optical flow and would otherwise yield almost no keyframes.
- **A ceiling from the preset.** Too many keyframes and matching time explodes, since the work grows faster than the number of frames.

There is also a special case in the pipeline: if there are between 2 and 80 frames total, *all* of them are kept. The reasoning is that such an input is not really a video but a sparse photo set — like an aerial mapping grid — where every frame is already a valuable wide-baseline view and thinning would be destructive.

11.7 Stages 7 to 14

The remaining stages each get a full part of their own: masking is Part 13, the SfM engine is Part 12, densification is Part 14, georegistration was Part 8, fusion and meshing are Part 15, the report is Part 17, and exports are Part 18. What follows here is only how the pipeline glues them together.

After SfM, the pipeline checks that the reconstruction produced any points at all and raises a clear error if not. It then applies the Sim(3) alignment to the points, the camera centres and (if present) the AI-densified points — all with the same transform, so everything stays in one consistent frame. It levels the scene. It fuses and cleans. It attempts meshing inside a `try` block, so a meshing failure adds a warning rather than losing the whole run. It builds the report, checks for a ground-truth file sitting next to the video and evaluates against it if found, then writes every artefact and a `manifest.json` recording the version, the video's SHA-256, the exact parameters used, all warnings, and — explicitly — `"coordinate_frame": "local ENU (metres) about telemetry origin"` and `"units": "metres"`.

THE POINT

The pipeline's job is sequencing, timing, progress reporting and never losing work to a recoverable failure. Optional stages are wrapped so they can fail without taking the run down; every assumption the code had to make becomes a warning attached to the result; and the manifest records exactly what was run on exactly which file, so a result can be traced back later.

PART 12

Inside the reconstruction engine

This is `sfm.py`, the file that actually turns pictures into 3D. Part 5 explained the ideas; this explains what this specific code does with them.

12.1 Step 1: finding features

Each keyframe is converted to greyscale and passed to **SIFT** — the Scale-Invariant Feature Transform, the classic and still highly respected feature detector. SIFT finds distinctive blob-like patches and describes each one with 128 numbers, in a way designed to stay similar when the patch is seen larger, smaller, rotated, or under different lighting. Up to 4,000 features are taken per frame.

The contrast threshold is set to 0.02, lower than the default. That makes SIFT accept fainter features — which matters for aerial imagery, where large areas of road, roof and field are low in contrast and would otherwise yield almost nothing.

If dynamic masking is on, the mask is handed to SIFT directly, so no feature is ever detected on a moving object in the first place.

12.2 Step 2: matching, and which pairs to try

Matching every frame against every other would be far too slow. The code builds a deliberate list of pairs:

- **Consecutive pairs** — frame 1 with 2, 2 with 3, and so on.
- **Skip-one pairs** — 1 with 3, 2 with 4. This widens the baseline, improving triangulation angles.
- **Spatial neighbours** — if GPS positions are available, a **k-d tree** (a data structure for fast nearest-neighbour lookup) finds each frame's seven closest neighbours *in space* and adds those pairs too.

That third category is a genuinely important addition, and the comment says why: it "makes the matcher robust to non-sequential capture orders (e.g. lawn-mower aerial mapping grids), not just continuous video passes." In a grid flight, the frame physically next to you may have been taken two minutes ago on the previous pass. Time-based pairing would miss it entirely; space-based pairing catches it.

The ratio test

For each pair, features are matched by brute force — every descriptor in one image compared against every descriptor in the other. But rather than just taking the closest match, the code applies **Lowe's ratio test**: find the two closest candidates, and accept the match only if the best is less than 0.75 times as far away as the second best.

EVERYDAY COMPARISON

Imagine identifying someone from a photo. If the best candidate is a strong resemblance and the second-best is nothing like them, you are probably right. If the best two candidates look equally similar, you should not commit — you are as likely to have picked the wrong one. The ratio test encodes exactly that reasoning, and it is remarkably effective at killing false matches in repetitive scenes like tiled roofs or crop rows.

Geometric verification

Surviving matches are then checked against the geometry. The code computes the **essential matrix** — a compact description of how two cameras with known intrinsics must be related — using RANSAC at 99.9% confidence with a 1-pixel threshold. Matches consistent with a single plausible camera movement are kept; the rest are discarded. Pairs producing fewer than 20 good matches are dropped entirely as unreliable.

12.3 Step 3: building tracks with union-find

The same physical point may be seen in ten different frames, producing nine separate pairwise matches. To triangulate properly you want all ten sightings grouped as one **track**.

The code does this with a **union-find** (or disjoint-set) structure — an elegant classic algorithm for grouping things that are connected. Each detected feature starts in its own group; each confirmed match merges two groups; at the end, everything transitively connected is in one group, and that group is the track. The implementation includes **path compression**, a standard optimisation that flattens the lookup chains and makes it run in effectively constant time.

Only tracks seen in at least two frames survive — a point seen once cannot be triangulated at all.

12.4 Step 4: getting started (two-view initialisation)

The reconstruction needs a seed. The code picks the *consecutive* pair with the most verified matches, recovers the relative rotation and translation from the essential matrix, and declares the first camera to be the origin with no rotation. Everything else in the reconstruction is expressed relative to that arbitrary choice — which is fine, because Part 8's alignment will later move the whole thing onto the map.

12.5 Step 5: triangulation, with two rejection tests

The function `_triangulate_track` takes all the sightings of one track and solves for its 3D position using the **Direct Linear Transform** — building a matrix of constraints from every observation and finding the solution via SVD. This uses *all* observations at once, not just two, so a point seen in eight frames is better determined than one seen in two.

Then two independent checks, and this is where fabricated geometry gets rejected:

- **The cheirality check.** The computed point is transformed into each camera's own frame, and its depth is required to be positive — the point must be *in front of* the camera. The mathematics of triangulation can happily produce a solution behind the camera, which is physically impossible: the camera cannot photograph what is behind it. Any such point is thrown away outright.
- **Reprojection and angle limits.** The point is projected back into every camera and the average error computed; if it exceeds 4 pixels, the point is rejected. The maximum angle between any two viewing rays is computed; if it is below 1° , the point is rejected as too poorly determined to trust.

THE POINT

This is the concrete meaning of the claim that nothing is fabricated. A point enters the cloud only if it was seen in at least two frames, sits in front of every camera that saw it, projects back to within 4 pixels of where it was actually detected, and was viewed from at least 1° apart. Points failing any of those are discarded, not downgraded.

12.6 Step 6: registering the remaining cameras

With a starting cloud in place, the remaining frames are added one at a time. For each, the code collects the features in it that belong to tracks already placed in 3D, giving pairs of "known 3D position" and "where it appears in this image". With at least six such pairs it solves for the camera's pose using **PnP** — Perspective-n-Point — wrapped in RANSAC with a 4-pixel tolerance, 200 iterations and 99.9% confidence, so a few bad correspondences cannot corrupt the pose.

Every fourth camera, the code re-triangulates all tracks, so newly added viewpoints improve existing points. That interval is a deliberate compromise, and the comment says so: re-triangulating after every camera would be cleaner but scales badly with the number of tracks.

CAREFUL HERE — THE HONEST GAP IN THIS ENGINE

What this engine does **not** do is global bundle adjustment. Part 5.5 described bundle adjustment as the step that refines every camera and every point together to minimise total error. This engine approximates it with repeated re-triangulation, which improves the points but never goes back and refines the *camera poses*.

The consequence is **drift**: small pose errors accumulate along the sequence, and a long flight can bend gently. This is precisely the gap COLMAP fills, and the COLMAP adapter's own comment names it — "COLMAP performs global bundle adjustment, which is what real wide-baseline aerial grids need." The trade was deliberate: an engine that always runs on any machine beats a better engine that often will not run at all. But it is a real limitation and worth being able to state plainly if asked.

12.7 Step 7: assembling the cloud

Every surviving point gets a colour, sampled from the actual pixel in the first registered frame that saw it (with a conversion from OpenCV's blue-green-red byte order to the red-green-blue that everything else expects). It gets its confidence from the formula in Part 9.3. And the engine returns summary statistics: how many keyframes registered and what fraction, how many points, the median and 90th-percentile reprojection error, the mean track length and the mean triangulation angle. Those numbers flow straight into the report and onto the screen.

PART 13

Removing things that move

Everything in Part 5 assumes the world holds still while the camera moves. Cars and people do not. This part is about detecting them — and about why this feature is switched off by default, which is one of the more instructive decisions in the project.

13.1 Why moving objects break reconstruction

Triangulation works by intersecting rays from two camera positions. That intersection is only meaningful if the thing being looked at stayed put between the two shots. If a car drove twenty metres between frame 10 and frame 40, the two rays are pointing at two *different places*, and where they happen to cross is a location no object ever occupied.

Worse, these phantom points do not just add junk. They feed back into the camera pose estimation, where the software tries to find camera positions consistent with all its observations — and inconsistent observations pull the cameras themselves off course. A busy road can degrade the entire reconstruction, not just the part with cars in it.

13.2 The default method: optical-flow residual

The default detector needs no AI model and no downloads. It works on a simple observation: as the drone moves, *everything* in the image appears to shift, and that shift is mostly explained by the camera's own motion. Anything moving independently will shift differently.

The procedure in `OpticalFlowMask`:

1. Compute **dense optical flow** between consecutive frames — for every pixel, how far it appears to have moved.
2. Sample that flow on a grid every 16 pixels and fit a single **partial affine transform** to it, using RANSAC. This is the global motion: the shift, rotation and zoom that the camera's own movement produced.
3. Subtract the prediction from the actual flow. What remains is the **residual** — the part of each pixel's movement that the camera's motion cannot explain.
4. Flag pixels whose residual exceeds 6 pixels.
5. Discard flagged regions smaller than 0.2% of the image, using connected-component analysis.
6. **Dilate** what remains by 9 pixels — grow the mask outward — so the blurry boundary around a moving object is also excluded.

EVERYDAY COMPARISON

Look out of a moving train. Everything streams past at a rate the train's speed explains. A bird flying alongside does not match that pattern, and it is the mismatch, not the movement, that makes it stand out. Steps 2 and 3 are "work out what the train's speed explains, then look at what is left over."

13.3 Why it is off by default

The pipeline's default is `mask_backend = "none"`, and the comment above it explains: *"the light-weight optical-flow residual method can conflate scene parallax with motion for tall structures."*

The failure is genuinely subtle. **Parallax** — the whole basis of reconstruction from Part 4 — means that near objects appear to shift more than far ones as the camera moves. So the top of a tall building, being closer to a low-flying drone than the ground around it, shifts *more* than the global motion model predicts. The residual is large. The masker concludes it is moving and paints out exactly the structure you most wanted to reconstruct.

CAREFUL HERE — THE SAMPLE DATA PROVES IT

This is not a theoretical concern. The repository ships two saved runs of the identical synthetic dataset, differing only in this setting. With optical-flow masking **on**, 16 of 29 keyframes registered and the building's depth was measured 57% wrong. With masking **off**, all 29 registered and the same measurement came out 0.6% wrong. Part 24 lays out both result sets in full. The mitigations in the code — the raised threshold, the minimum blob size — reduce the problem but do not eliminate it, and turning the feature off by default was the correct call.

13.4 The optional semantic method

The better approach is to identify moving objects by *what they are* rather than by how they move. `TorchvisionSemanticMask` uses **semantic segmentation** — a neural network that labels every pixel with an object category — and masks out the categories that can move: people, animals, vehicles.

The model is DeepLabV3 with a MobileNetV3 backbone, a good speed-to-accuracy compromise. It uses the graphics card automatically when one is present, and the environment variable `DRISHTI_DEVICE` can force it either way.

CAREFUL HERE — THIS PATH NEEDS WEIGHTS YOU MUST SUPPLY

The model is constructed with `weights=None`, deliberately, so that nothing is downloaded from the internet. A neural network without trained weights is random — it will produce meaningless masks. To make this backend actually useful you must point `DRISHTI_SEG_WEIGHTS` at a local checkpoint file. This is the honest consequence of the offline requirement, but it does mean that simply selecting "semantic" in the interface without supplying weights gives you a masker that does nothing sensible.

If Torch is not installed at all, `get_backend` falls back to optical flow and returns the warning *"torchvision unavailable; using optical-flow residual masking instead"*, which is surfaced to the user. Never a crash.

PART 14

Filling the holes with AI, honestly

Classical reconstruction leaves holes wherever there was no texture to match — plain roads, flat roofs, water. An AI depth model can fill them convincingly. The interesting engineering here is not the filling; it is the machinery that keeps the filled parts from ever being mistaken for measurements.

14.1 The problem, stated exactly

The file's own comment calls this "the single-pass moat" and describes the situation precisely: a single linear fly-over gives classical SfM tiny parallax and no loop closure, so textureless regions fail feature matching and the cloud comes out sparse and full of holes.

The result is a reconstruction that is accurate everywhere it exists but exists in surprisingly few places — points clustered on building edges, road markings and vegetation, with large empty areas between them.

14.2 Monocular depth estimation

A **monocular depth model** is a neural network trained on enormous quantities of images paired with known depth, which learns to estimate depth from a *single* image. It is doing what your brain does when you look at a photograph and perceive depth without any stereo information: reading perspective, texture gradients, occlusion, familiar object sizes, haze.

Drishti3D uses **Depth Anything V2** (the "Small" variant by default), a well-regarded recent model, loaded through the Hugging Face `transformers` library.

CAREFUL HERE — AN INFERENCE, NOT A MEASUREMENT

A monocular depth model has no geometric evidence whatsoever. It is pattern-matching against everything it has previously seen. It is often impressively right and occasionally confidently wrong, and nothing in its output distinguishes the two cases. This is exactly the "AI may create a plausible completion" the original brief warned about — which is why every point it contributes is labelled AI-assisted.

14.3 The clever part: anchoring the AI to real geometry

A monocular model's output is **affine-invariant**: the *relative* depths are meaningful — this is nearer than that — but there is no scale or offset. It cannot tell you anything is 40 metres away.

The solution in `depth_prior.py` is neat. For each keyframe:

1. Take the sparse SfM points that are visible in this frame. These have **real, metrically-scaled depths** from actual triangulation. Call them **anchors**. At least 20 are required.
2. Run the depth model on the frame to get its relative depth map.
3. Look up the model's value at each anchor's pixel. Now you have pairs: what the model said, and what is actually true.
4. Fit a straight line relating the two — a scale and an offset — so the model's output is stretched onto the real metric scale.
5. Apply that fitted relationship to every pixel and project the results out into 3D.

So the AI supplies the *shape* of the depth variation across the image, while the classical reconstruction supplies the *scale*. The AI never invents a size; it interpolates between real measurements.

EVERYDAY COMPARISON

Like a sketch artist who is excellent at proportions but has no ruler. Give them two known dimensions on the drawing and every other dimension becomes calculable. The two known dimensions are the sparse SfM points; the proportions are the AI's depth map.

Robust fitting

The line-fitting uses **iteratively reweighted least squares** with Cauchy weights — three rounds where points that disagree strongly with the current fit get progressively less influence. This is a gentler cousin of RANSAC: instead of a hard in-or-out decision, badly-fitting anchors simply fade out. It matters because some anchors will be wrong, and one bad anchor in a plain least-squares fit would tilt the entire frame's depth.

Refusing to extrapolate

After back-projection, the code discards any point whose depth is less than half the closest anchor or more than 2.5 times the furthest. Beyond that range the AI is guessing outside anything the real geometry vouched for, and the fitted relationship has no support. Additional limits cap the output at 45,000 points per frame and 500,000 overall, sampling a pixel grid every 8 pixels rather than using every pixel.

14.4 Keeping it separate, permanently

The pipeline never merges these points into the observed cloud. They are voxel-downsampled at a coarser resolution, cleaned of stray outliers, and then added by `add_inferred_layer` which sets their confidence to exactly zero and their provenance to `AI_ASSISTED`. From there, Part 9's four layers of enforcement take over: measurements skip them, the viewer draws them purple and can hide them, and the exports carry the label.

The interface says the same thing in plain words when you select the option: *"Added points are tagged AI-assisted — shown as a distinct trust-map layer and excluded from measurements by default."*

THE POINT

This is the project's thesis made concrete. The AI is genuinely useful — it turns a sparse skeleton into something that reads as a surface — and it is anchored to real measurements rather than guessing freely. And it is quarantined so completely that you have to deliberately opt in before a measurement can touch it, at which point the result carries a permanent warning.

PART 15

Cleaning up and building a surface

Raw reconstruction output is noisy and unevenly dense. Two files tidy it: `fusion.py` cleans and labels, `mesh.py` optionally turns it into a solid surface.

15.1 Voxel downsampling

Points arrive unevenly — thousands crowded onto a well-textured wall, almost none on a plain road. **Voxel downsampling** evens this out. Space is divided into a grid of cubes (a **voxel** is a 3D pixel), here 0.15 m on a side by default, and each occupied cube keeps one representative point.

This is not merely a size reduction. It caps the resolution at something honest — you are declaring that the model resolves detail at about 15 cm and no finer — and it makes downstream operations behave predictably, since surface-normal estimation and meshing both work far better on uniformly spaced points.

15.2 Statistical outlier removal

Some points are simply wrong: a mismatched feature that survived every check, a reflection, a point on a bird. They tend to float in empty space away from everything else.

The removal is a nice piece of statistics. For every point, measure the average distance to its 12 nearest neighbours. Across the whole cloud, compute the mean and standard deviation of that quantity. Any point whose neighbours are further away than the mean plus two standard deviations is deleted. Points on a real surface have close neighbours; isolated points do not.

EVERYDAY COMPARISON

Like finding people standing on their own at a crowded party. You do not need to know where the groups are — you just measure how far each person is from their nearest few neighbours, and the ones with unusually large distances are the loners.

15.3 Normals

A **normal** is the direction a surface faces at a point — perpendicular to it, like a tiny arrow sticking out. Points alone carry no orientation, but you can infer it: look at a point's neighbours, fit the plane they lie in, and take the perpendicular to that plane.

Normals are needed for two reasons. Meshing algorithms require them to know which side of a surface is outside. And lighting a 3D model requires them, because how bright a surface appears depends on its angle to the light.

15.4 The Open3D dependency, and the fallback

Open3D is a well-known library for 3D data. Drishti3D uses it for downsampling, outlier removal, normals, plane fitting and meshing — but treats it as optional throughout. `fusion.py` contains a complete pure-NumPy alternative for downsampling and outlier removal, so the cleanup still happens without it. Meshing genuinely requires it, and if it is absent the code raises a message that says so and explicitly reassures: *"Point cloud output is preserved."*

15.5 Poisson surface reconstruction

Meshing uses **Poisson surface reconstruction**, the standard modern method. The intuition: treat each oriented point as a small piece of evidence about where the inside/outside boundary of a solid object lies, then solve for the smooth surface that best satisfies all that evidence at once. It is a global solve, which is why it handles noise well and produces smooth watertight results.

The **depth** parameter (9 here) controls the resolution of the underlying grid, doubling detail and cost with each increment.

CAREFUL HERE — POISSON INVENTS SURFACES, BY DESIGN

Poisson reconstruction produces a *closed* surface. Faced with a hole where the drone saw nothing, it will confidently span it — and that spanned region looks exactly like a real measured wall. This directly contradicts the project's central principle.

The code's defence is **density-based cropping**: Poisson reports, for each vertex it produced, how much actual point evidence supported it, and the code deletes the bottom 10% — the vertices that were mostly invention. This helps considerably but is a threshold, not a guarantee. The correct conclusion, and the one the project takes, is that the **point cloud is the authoritative output and the mesh is a visualisation**. If someone asks whether the mesh can be trusted for measurement, the answer is that measurements are computed on the cloud, not the mesh.

PART 16

Measuring things

The whole point of the exercise. Four measurement types, each with the trust rules built in.

16.1 Snapping

Every measurement begins with a click in the 3D viewer, and every click begins with **snapping**. Your click gives an approximate 3D location; the code finds the nearest actual point in the cloud and uses that instead.

This matters for two reasons. It makes measurements repeatable — click near the same corner twice and you get the same point, so you get the same answer. And it is where the provenance filter lives: unless you have opted in, AI-assisted points are removed from the search set entirely *before* the nearest point is found. You cannot snap to inferred geometry by accident, because as far as the search is concerned it is not there.

16.2 The four measurements

Type	Clicks	What it computes
Point	1	The coordinates of one place, in local ENU metres, convertible to latitude and longitude.
Distance	2+	The straight-line 3D distance between consecutive points, added up. With more than two clicks it becomes a path length — useful for following a road or a wall.
Height	2	The difference in the "up" coordinate only, ignoring horizontal separation entirely. So you can click the base of a building and any point on its roof, and get the height even if the two are far apart horizontally. This is the measurement that depends on the levelling from Part 8.7 — if the model is tilted, "up" is not up.
Area	3+	The area of the polygon you clicked out. Because the points will not be exactly coplanar, the code fits the best plane through them using SVD, projects them onto it, and applies the shoelace formula — a classic method that computes a polygon's area from its corner coordinates directly.

16.3 What gets recorded

Every measurement is stored with more than its number: the kind, the value, the unit (always metres or square metres), the actual snapped coordinates, a confidence note, whether inferred geometry was used, any warnings, and a timestamp. The interface shows the value alongside its note and warnings, and the value never appears without them.

THE POINT

A measurement here is not a bare number. It is a number plus the coordinates it came from, plus a statement of how trustworthy the geometry underneath it was. That package is what gets stored, displayed and exported — which is exactly what you would want if the number ever had to be defended.

PART 17

The quality report

The report is the project's evidence. Here is every number in it and what it actually tells you — including which ones are easy to over-read.

17.1 Input section

Video resolution, frame rate, duration, codec, and the **SHA-256**. Telemetry sample count, any parse warnings, and whether RTK was detected. Plus `scale_source`: `rtk`, `gps`, or `relative` — the single most important field for interpreting everything else, because it tells you which accuracy regime you are in.

17.2 Frames section

How many frames were analysed, accepted, rejected, and how many became keyframes. A low acceptance rate points at the capture: blurry footage, bad light, or a hovering drone producing duplicates.

17.3 Reconstruction section — the diagnostic core

Number	How to read it
Registered keyframes (and the fraction)	How many keyframes the engine successfully placed. The single most informative number in the report. Near 100% means the reconstruction went well. Around 50% means half your footage contributed nothing, and any other number should be read with suspicion.
Median reprojection error	Typical disagreement, in pixels, between where a point should appear and where it was detected. Below about 1 px is good. Above 2–3 px indicates bad intrinsics, bad matches, or motion blur. The 90th percentile is also reported, showing the tail.
Mean track length	How many frames the average point was seen in. Higher is better: a point seen in 8 frames is far better determined than one seen in 2. Values near 2 mean the reconstruction is only just holding together.
Mean triangulation angle	The average intersection angle of the viewing rays, in degrees. Larger means better-conditioned depth. Small values are the signature of the single-pass problem.

17.4 Cloud section

Point count; the bounding box and overall dimensions in metres (a good sanity check — if a suburban block comes out 4 km across, something is wrong); the **median point spacing**, which is

the honest statement of the model's resolution; and the counts and fractions in each provenance class.

17.5 Alignment section

Scale factor, inliers out of total, and horizontal and vertical RMSE — all as described in Part 8.6. And attached to every one of them, in the JSON, in the HTML and on screen, the note that this is **consistency, not accuracy**.

17.6 Performance section

Total processing time, per-stage timings, and the **processing-to-video ratio** — processing seconds divided by video seconds. A ratio of 7 means a two-minute flight takes fourteen minutes to process. This is the honest efficiency figure the original brief asked for.

17.7 Ground-truth section — the only real accuracy figures

Present only when a `ground_truth.json` sits beside the video, which is the case for the bundled synthetic dataset. This is the section that contains genuine accuracy, because it compares against something independent:

- **Surface accuracy** — for every reconstructed point, the distance to the nearest true surface. Reported as median, mean, 90th percentile and RMSE.
- **Completeness at 0.5 m, 1 m and 2 m** — the fraction of the true surface that has a reconstructed point within that distance. This is the complement of accuracy: accuracy asks "is what you built correct?", completeness asks "how much of what exists did you build?" A tiny but perfect fragment scores perfect accuracy and terrible completeness.
- **Dimensional accuracy** — the system measures known reference distances on its own reconstruction and compares to the true values, reporting absolute and percentage error. **This is the number that matters most**, because it is exactly what a user would do with the product.

17.8 Warnings and limitations

Every warning accumulated during the run, and a fixed list of limitations that is included in *every* report regardless of how well the run went:

PRINTED ON EVERY REPORT, ALWAYS

- Monocular single-pass video cannot measure surfaces never observed.
- Absolute accuracy is GPS-limited unless RTK/PPK or GCPs are provided.
- AI-assisted geometry is excluded from measurement by default.
- Alignment residual is not a certified accuracy statement.

The HTML version is a self-contained printable page with no external files, so it can be emailed, archived or printed and will still render identically years later — including a footer stating it is not an official NTRO product and that the figures reflect only that run's inputs.

PART 18

Exports: the file formats

A result trapped inside one application is not much use. Seven formats, each for a different destination.

Format	Opens in	What it is and what is special here
PLY <code>point_cloud.ply</code>	CloudCompare, MeshLab, Blender	The Polygon File Format — the standard, simple way to store a point cloud. Written as plain readable text. Extended here with two extra per-point properties beyond position and colour: a <code>confidence</code> float and a <code>provenance</code> code. So the trust labelling survives into other software.
LAS <code>point_cloud.las</code>	QGIS, ArcGIS, survey software	The LiDAR industry's standard format, and what geospatial professionals expect. Confidence is stored in the intensity field, since LAS has no confidence field of its own. Requires the <code>laspy</code> library; if it is missing the export is skipped with a warning and everything else still works.
GLB <code>mesh.glb</code>	Blender, web viewers, Windows 3D Viewer	The binary form of glTF, the modern standard for shipping 3D models — geometry, colours and structure in a single file. Only written if meshing succeeded.
GeoJSON <code>trajectory.geojson</code>	QGIS, Leaflet, GitHub	The web standard for geographic data. Contains the flight path as a line in real latitude and longitude, plus any measurements. Note the coordinate order is longitude first, then latitude — a genuine trap, since everyone says "lat, long" in speech but the format specifies the reverse. The code has it right.
CSV <code>trajectory.csv</code>	Excel, Python, anything	The reconstructed camera positions as a plain table: frame index, east, north, up. The universal fallback that every tool on Earth can read.
JSON <code>quality_report.json</code>	Any program	The complete report as structured data, for automated comparison across runs.
HTML <code>quality_report.html</code>	Any browser	The same report as a printable page. Self-contained.

Two additional files are written for internal use: `cloud.npz`, a compressed binary NumPy archive holding the full-resolution cloud (this is what the measurement endpoint loads), and `viewer.json`, a browser-friendly payload downsampled to at most 120,000 points with coordinates rounded to millimetres — because sending several million points to a web page would be unusable.

THE POINT

Every export states its coordinate frame and units, and the provenance labelling is carried into the two formats that can hold it. Optional formats degrade with a warning rather than failing the run. The 120,000-point limit on the viewer payload is why the browser display stays responsive.

PART 19

The backend

Everything so far has been a Python library. The backend is what turns it into something a person can use from a browser: a web server, a database, a job runner, and a set of security rules.

19.1 What a backend is

When you use a web application, two programs are involved. The **frontend** runs inside your browser and draws everything you see. The **backend** runs on a server, holds the data, and does the heavy work. They talk to each other over the network by exchanging small structured messages.

Here the split is natural: reconstruction needs a real processor and access to files, and cannot happen in a browser. So the browser sends "please process this project" and the backend does the fourteen stages.

19.2 FastAPI

FastAPI is the Python framework used. Its distinguishing feature is that you describe the shape of your data in ordinary Python type annotations, and it then validates every incoming request against that description automatically, rejecting malformed input with a clear error before your code ever sees it. It also generates interactive documentation for the whole API from the same descriptions — visit `/docs` on the running server and you get a browsable, clickable, try-it-yourself page for every endpoint, for free.

The data shapes live in `schemas.py` and use **Pydantic**, the validation library FastAPI is built on. For example, a project's name is declared as a string of between 1 and 200 characters — so an empty or absurdly long name is rejected automatically, in one line, with a helpful message.

19.3 The database

Three tables, defined in `models.py` using **SQLAlchemy** (a library that lets you work with database rows as ordinary Python objects instead of writing SQL by hand):

- **Project** — one drone mission. Name, description, status, creation time, the uploaded filenames, the video's SHA-256, and any intrinsics.
- **Job** — one processing run of a project. Status, current stage, progress, message, error text, warnings, timestamps.
- **Measurement** — one measurement taken on a result. Kind, value, unit, coordinates, confidence note, whether inferred geometry was used, warnings.

The database itself is **SQLite** — a complete relational database that is just a single file on disk, with no server to install or run. Perfect for this: it makes the whole application self-contained and

air-gap friendly. Because SQLAlchemy sits in between, moving to **PostgreSQL** with the **PostGIS** geographic extension later would be a configuration change rather than a rewrite, which is what the original plan called for.

Deleting a project uses a **cascade**, so its jobs and measurements go with it, and then the files on disk are removed too — scoped strictly to that project's own directory.

19.4 Background jobs

A reconstruction takes minutes. An HTTP request that took minutes to answer would time out. So `jobs.py` runs the work in the background: the request returns immediately with a job identifier, and the actual processing happens on a separate thread.

The thread pool is deliberately sized at **one worker**, with the comment "serialise heavy CV jobs". Reconstruction is processor-hungry; running two at once would make both slower and could exhaust memory. Jobs queue instead.

Progress is kept in two places: an in-memory dictionary updated on every callback, and the database, written at most twice a second. The reason for the split is that writing to the database thousands of times during a run would be a real drag on performance, while the in-memory copy can be updated freely and is what the live progress stream reads. The dictionary is protected by a lock, since two threads touch it.

If anything fails, the exception is caught, the full traceback is printed to the server log, the job is marked failed with the error message, the project is marked failed, and the message reaches the user's screen. A failed reconstruction never takes the server down.

19.5 Live progress: Server-Sent Events

How does the browser learn about progress? Not by asking repeatedly. The endpoint `/api/jobs/{id}/events` uses **Server-Sent Events** (SSE) — the browser opens one connection and holds it open, and the server pushes updates down it whenever something changes.

The implementation checks the in-memory state four times a second and sends a message only when it has actually changed, then closes the stream when the job finishes or fails. The browser side handles a dropped connection by falling back to a single direct query for the final state, so a network hiccup cannot leave the interface stuck at 60% forever.

THE WORD PEOPLE ACTUALLY USE

SSE is one-way: server to browser. The alternative, **WebSockets**, is two-way but more complex. Since progress only ever flows one direction, SSE is the simpler correct choice.

19.6 Security

Accepting file uploads from users is one of the more dangerous things software can do. `storage.py` is short and every line of it is a defence:

Threat	The defence in the code
Path traversal — a filename like <code>../../../../etc/passwd</code> designed to write outside the intended folder	Any directory part of the name is stripped, then everything except letters, digits, dot, underscore and hyphen is replaced. Leading dots are removed. The name is truncated to 200 characters. There is a test asserting this specific attack fails.
Forged project id in the URL, aiming at another directory	The id must match exactly 32 hexadecimal characters — the shape of a server-generated identifier. Anything else is rejected immediately, with its own test.
Wrong file type	Videos must be <code>.mp4 .mov .m4v .avi .mkv</code> ; telemetry must be <code>.csv .json .srt</code> . Anything else is refused with a message listing what is allowed.
Enormous upload filling the disk	The file is streamed in one-megabyte chunks with a running size check. Exceeding the limit (2 GB by default) aborts <i>and deletes the partial file</i> . Empty files are refused too.
Executing uploaded data	Never happens. Uploads are only ever read as video or text.
Unwanted network exposure	The server binds to <code>127.0.0.1</code> by default — reachable only from the same machine. Making it visible on a network requires a deliberate configuration change.

Every upload is hashed as it is written, and the originals are treated as immutable — the pipeline reads them and writes its outputs elsewhere.

19.7 The honest capabilities endpoint

One endpoint deserves singling out. `GET /api/capabilities` probes what is genuinely installed on this machine right now — whether PyCOLMAP imports, whether Open3D is present, whether laspy is present, whether Torch is present, and whether each AI adapter reports itself available — and returns the truth. The dashboard displays it as a row of indicator pills with the caption: *"Availability is probed live — nothing is claimed that is not installed."*

This is a small feature that does a lot of work in a demonstration. It turns "we support COLMAP" from a claim into something a sceptic can check on the spot.

PART 20

The API, endpoint by endpoint

An API is a list of specific requests a program can make. Each has a method (what kind of action) and a path (what it acts on).

THE WORD PEOPLE ACTUALLY USE

GET means "give me this" and changes nothing. **POST** means "here is something, create or do it". **DELETE** means "remove this". Anything in `{braces}` is a placeholder you fill in with a real identifier. This convention is called **REST**.

Method	Path	What it does
GET	/api/health	Answers "are you alive?". Used to check the server is up.
GET	/api/capabilities	Reports which engines and optional libraries are really installed.
Projects		
POST	/api/projects	Create a new mission from a name and description.
GET	/api/projects	List all missions, newest first.
GET	/api/projects/{id}	Get one mission's details.
DELETE	/api/projects/{id}	Delete a mission, its database rows and its files.
POST	/api/projects/{id}/video	Upload the drone video.
POST	/api/projects/{id}/telemetry	Upload the telemetry file.
POST	/api/projects/{id}/intrinsic	Supply the camera's fx, fy, cx, cy.
Processing		
POST	/api/projects/{id}/process	Start a reconstruction. Takes the preset, masking choice, meshing choice, engine, densification setting and optional intrinsic. Returns a job immediately.
GET	/api/jobs/{id}	Get a job's current status, preferring the live in-memory state.
GET	/api/jobs/{id}/events	The live progress stream (SSE).
Results		
GET	/api/projects/{id}/quality	The full quality report as JSON.
GET	/api/projects/{id}/trajectory	The GPS track and the reconstructed camera path.
GET	/api/projects/{id}/model	The point cloud, downsampled for the browser, with provenance.
GET	/api/projects/{id}/keyframes	Which frames were chosen as keyframes.
GET	/api/projects/{id}/frame_metrics	Every frame's quality scores and rejection reasons.
GET	/api/projects/{id}/exports	Which export files actually exist for this project.
GET	/api/projects/{id}/exports/{key}	Download one of them.
Measurements		
POST	/api/projects/{id}/measurements	Take a measurement: the kind, the clicked points, and whether inferred geometry is allowed.
GET	/api/projects/{id}/measurements	List all measurements taken on this project.

Method	Path	What it does
DE-LETE	<code>/api/.../measurements/{mid}</code>	Delete one.

Two implementation notes. The exports endpoint lists only files that *exist*, so the interface can never offer a download that would fail — if meshing was skipped, no GLB button appears. And the measurement endpoint caches the loaded point cloud in memory, since parsing a multi-mega-byte cloud file for every single click would make measuring feel sluggish.

PART 21

The screens the user sees

A React application with five views and a 3D viewer written directly against the browser's graphics system.

21.1 The technology, briefly

- **React** — the library for building interfaces out of reusable components, where you describe what the screen should look like for a given state and the library works out how to update the display.
- **TypeScript** — JavaScript with types. Every piece of data from the backend has a declared shape in `api.ts`, so if the backend changes and the frontend does not, the error appears at build time rather than as a blank screen for a user.
- **Vite** — the build tool and development server. It also **proxies** `/api` to the backend during development, which neatly sidesteps browser cross-origin restrictions.
- **Three.js** — the standard library for 3D graphics in a browser, wrapping WebGL.

Notably absent: **CesiumJS** and **MapLibre**, both of which the original plan proposed for globe and map display. The finished application uses Three.js for 3D and a small hand-written SVG plot for the trajectory map. That is a real simplification with a real benefit — a hand-drawn SVG plot needs no map tiles, which means **it works with no internet connection**, which was a stated requirement. A Cesium globe would want tile data from somewhere.

21.2 The five views

Dashboard

Lists all missions as cards with status badges, whether video and telemetry are present, and links onward. Above them, the live capabilities strip from Part 19.7. Clicking a mission goes to the workspace if it is finished, or the monitor if it is still running.

Wizard — new reconstruction

Four steps: name the mission, upload the video, upload the telemetry, then set camera and processing options. Each step is gated on the previous one succeeding, so you cannot start processing without the inputs. The options screen exposes the preset, the masking backend, densification, meshing, and optional intrinsics — and it shows contextual notes, including a warning when you select optical-flow masking that it "may flag scene parallax on tall structures; best for near-nadir passes." The software warns you about its own weak spot at the moment you might trigger it.

Monitor

Opens the SSE stream and shows the current stage, its message, a percentage, elapsed seconds, and the full fourteen-stage timeline with completed stages green, the current one highlighted and future ones grey. Warnings accumulate in a panel below. On failure it shows the actual error text rather than a generic apology.

Workspace — the main screen

Three columns:

- **Left** — the measurement tools, the "include inferred geometry" checkbox, the result of the last measurement with its confidence note and warnings, display controls (true colour versus provenance colour, splat mode, point size), and a checkbox per provenance class with its live point count.
- **Centre** — the 3D viewer, with a live coordinate readout in both ENU metres and latitude/longitude as you hover.
- **Right** — the key quality numbers, the trajectory plot, a keyframe timeline where each frame is a tick coloured green or red and outlined if it became a keyframe, the list of saved measurements, and the export buttons.

Report

The full evidence report, including provenance distribution drawn as proportional bars and, when ground truth is available, the dimensional accuracy table. With links to open the standalone HTML version or download the JSON.

21.3 Inside the 3D viewer

`PointCloudViewer.tsx` is the most technically dense file in the frontend. A few things in it are worth understanding:

- **Up is Z.** The camera's up vector is set to (0,0,1) because the data is in ENU where up is the third axis. Three.js normally assumes Y is up. Getting this wrong would display everything on its side.
- **Two colour arrays, precomputed.** Both the true colours and the provenance colours are built once at load. Switching between them is then instant, because it is a copy rather than a re-computation.
- **Hiding by NaN.** To hide a provenance class, its points' positions are set to "not a number". The graphics system silently skips them. It is a neat trick that avoids rebuilding the geometry every time you tick a checkbox.
- **Splat mode.** Optionally each point is drawn as a soft radial blob rather than a hard square, at about three times the size. Overlapping blobs blend into what reads as a continuous surface — a cheap approximation of a real render that makes a sparse cloud look far more like a photograph.

- **Picking by raycasting.** When you click, a ray is cast from the camera through your cursor into the scene, and the first point it hits is returned. The hit tolerance scales with the model's size, so picking feels the same on a small building and a large site.
- **Trackball controls.** Chosen over the more common orbit controls specifically so the model can be rotated freely in every axis including over the top, rather than being constrained to keep one direction upward.

PART 22

Proving it works: the evaluation harness

Everything in the report so far describes what the software thinks it did. The `eval/` package is about checking whether it was right, against data where the truth is known independently.

22.1 Why this exists

The alignment residual, as Part 8.6 laboured, is internal consistency. Real accuracy needs an external reference. The evaluation harness runs the full pipeline on datasets that carry ground truth and scores the result against it.

22.2 The rules it plays by

Three honesty rules are built into the design, and they are what make the numbers worth anything:

- **Ground-truth poses are never fed to the pipeline.** The pipeline gets a video and a telemetry file, exactly like a real user. It never sees the answers.
- **Datasets without GPS get realistic fake GPS, not perfect GPS.** Many research datasets have precise camera positions but no GPS track. Feeding the exact positions in as "GPS" would be cheating spectacularly. Instead the harness takes the true positions and *adds noise* — 2.5 m horizontally and 4 m vertically by default, which is genuine consumer-drone quality — then converts them to latitude and longitude. So the pipeline works under the same handicap it would face in the field.
- **Every spatial number is reported after a Sim(3) alignment.** Because a reconstruction has an arbitrary starting position, orientation and scale, comparing it directly to ground truth would measure that arbitrariness rather than the quality. Fitting the best similarity transform first removes it. This is the standard protocol in the SLAM literature — the harness calls it "gauge-free" — and it means a global offset can neither flatter nor unfairly penalise the result.

22.3 What it measures

Metric	What it is
ATE Absolute Trajectory Error	After alignment, the distance between each estimated camera position and its true position. Reported as RMSE, median, maximum, and separately per axis (east, north, up) — which is diagnostic, since a much larger vertical error is the expected signature of the constant-altitude problem from Part 8.7.
Scale error	How far the recovered scale is from 1.0 after alignment. The evaluation documentation calls this "the headline for the metric-safety wedge" — it directly answers whether real size was recovered from noisy GPS and monocular video in a single pass.
Cloud accuracy	For each reconstructed point, the distance to the nearest true surface. Median and RMSE.
Completeness	The fraction of true surface points with a reconstruction point within a threshold (0.5 m by default).
Dimensional error	Percentage error on known reference distances, taken from the pipeline's own ground-truth check.

The ICP and cropping detail

Scoring the cloud is done carefully, following the protocol used by the well-known ETH3D and Tanks-and-Temples benchmarks. The reconstruction is registered to the true cloud by **ICP** — Iterative Closest Point, which repeatedly finds each point's nearest counterpart and computes the transform that best aligns them — with **scale locked at 1**, so ICP cannot shrink the model to flatter the numbers. It is also **trimmed**, using only the best-fitting 80% each round, so outliers cannot drag it. Then the result is **cropped to the region the ground truth actually covers** before scoring.

That cropping matters more than it sounds. The comment explains: reusing the pipeline's own uncropped number "would count sky/ground outliers far outside the GT and inflate the error several-fold." A reconstruction legitimately extends beyond the region the ground truth measured; penalising it for that would be measuring the wrong thing.

22.4 The datasets

Dataset	What it is and why it is on the list
OpenDroneMap sample sets	Top priority, because it is <i>real drone imagery</i> — geotagged aerial photographs of actual neighbourhoods, fields and houses, with GPS in the EXIF metadata. About 120 such photos are already bundled in the repository. The closest thing to the real use case.
TartanAir	A large simulated dataset with perfect ground truth for depth, pose, segmentation and optical flow. Because it is synthetic, the truth is exact — ideal for isolating whether an error is the software's fault.
ETH3D	An academic SLAM and reconstruction benchmark with laser-scanned ground truth.
AGI2P / RTK-SLAM	Aerial imagery with airborne-laser-scanned point clouds and true poses.
Synthetic self-test	Generated by <code>synth.py</code> with no download at all. A textured ground plane and a building of exactly known dimensions, rendered from a moving camera. Because the geometry is exact by construction, accuracy can be computed rather than estimated.

The documentation also insists on a **held-out test set** — one dataset kept aside and never run during development, scored only once for the final credibility number. This is standard scientific discipline: if you tune against a dataset, your score on it stops being a prediction of performance on new data.

Running it is one command. `python -m eval.run_eval --self-test` needs no downloads; `python -m eval.run_eval --root eval_data` auto-detects the layout of whatever datasets you have placed there. Output is a markdown scorecard and a JSON report.

PART 23

Tests, Docker, and running offline

23.1 What is tested

Seven test files, run with `pytest tests/ -q`. Two kinds are represented: **unit tests**, which check one function in isolation, and an **integration test**, which runs the whole thing end to end.

File	What it verifies
<code>test_geo.py</code>	That converting coordinates and back returns the original (a round-trip test — the standard way to check a conversion). That east is east and north is north. That Umeyama exactly recovers a known transform. That RANSAC still recovers the right answer when four of twenty-five points are 40 metres wrong. That fewer than three correspondences is refused.
<code>test_telemetry_sync.py</code>	CSV, JSON and alias parsing. That an out-of-range latitude is skipped <i>and warned about</i> . RTK detection. That interpolating halfway between two GPS samples lands halfway. That yaw interpolation from 350° to 10° passes through 0°, not backwards through 180°.
<code>test_quality_measure.py</code>	That a noisy image scores sharper than a flat one. That dark and bright frames are flagged. That distance, height and area measurements are correct on a synthetic grid. And — importantly — that a measurement does not snap to an AI-assisted point when inferred geometry is not permitted.
<code>test_api_storage.py</code>	The security tests: that <code>../etc/passwd</code> is sanitised, that a project id of <code>../etc</code> is rejected.
<code>test_integration.py</code>	A complete pipeline run on a small generated fixture.
<code>test_eval.py</code>	The evaluation harness itself.
<code>conftest.py</code>	Shared setup: makes the packages importable and points the backend at a throwaway data directory, so tests never touch real projects.

The tests deliberately download no model weights, so they run on any machine and in any continuous-integration system.

23.2 Docker

Docker packages an application together with everything it needs — the operating system libraries, the Python version, every dependency — into an image that runs identically anywhere. It solves "it works on my machine".

Here `docker-compose.yml` defines three services: the backend on port 8000, the frontend served by nginx on port 8080, and an optional GPU worker that is disabled unless you explicitly ask for it. A named volume holds the data so it survives restarts. The command is `docker compose up --build`.

The backend image is built in a deliberate order — the reconstruction package first, then the backend requirements — so that changing backend code does not force a rebuild of the slow-to-install scientific libraries. Small detail, large effect on how pleasant the project is to work on.

23.3 Air-gapped operation

An **air-gapped** system is physically disconnected from any network. Intelligence and defence organisations often require it, so this was a real requirement rather than a nicety. What makes it work:

- The default pipeline makes **no network calls** during processing.
- Model weights are **never auto-downloaded**. Optional models require a local path in an environment variable. The semantic masker is explicitly constructed with `weights=None` to guarantee it.
- FFmpeg is bundled through a Python package rather than assumed to be installed.
- The trajectory map is drawn as a hand-made SVG, needing no map tiles from any server.
- The server binds to localhost by default.
- Deployment instructions cover building images on a connected machine, exporting them with `docker save`, and loading them on the isolated target.
- All state lives in SQLite plus one `data/` directory, so backup is a folder copy.

CAREFUL HERE — ONE EXCEPTION

The depth-prior densification is the one feature that *does* fetch from the internet: it downloads Depth Anything V2 weights from the Hugging Face hub on first use, as its own docstring says. On an air-gapped machine you must pre-download the model and point `DRISHTI_DEPTH_MODEL` at the local copy. Everything else genuinely runs disconnected; this one needs preparation.

PART 24

What the bundled sample runs actually show

The repository ships two complete saved runs of the same dataset. Comparing them teaches more about this system than any amount of description — and it is the kind of thing you want to have looked at before someone else does.

24.1 The setup

Both runs used `sample_data/synthetic/synthetic_flight.mp4`: a 2-second, 960×540, 60-frame synthetic flight generated by `synth.py`. Because the scene was rendered from known geometry, the true dimensions are exact: the building is **12 m wide, 8 m deep and 9 m tall**.

Both runs used the balanced preset, and both ran *without* camera intrinsics, so both carry the estimated-focal warning. The only meaningful difference between them is dynamic masking.

	Run A — <code>proj_demo</code>	Run B — <code>proj_none</code>
Dynamic masking	<code>optical_flow</code>	<code>none</code>
Keyframes registered	16 of 29 (55%)	29 of 29 (100%)
Points in final cloud	3,590	10,819

24.2 The measurements against known truth

Reference	True	Run A	Error A	Run B	Error B
Building width	12.00 m	10.81 m	9.9%	12.29 m	2.4%
Building depth	8.00 m	12.53 m	56.7%	8.05 m	0.6%
Building height	9.00 m	5.95 m	33.9%	9.31 m	3.5%

24.3 What this tells you

THE POINT

With masking off, the system measured a building it had never been told about to within **0.6% to 3.5%** of truth, from a two-second monocular video, on a CPU, with no camera calibration supplied. That is a genuinely good result and it comfortably meets the original brief's target of "below 3–5% relative dimensional error on well-observed surfaces."

With optical-flow masking on, the *same software on the same footage* got the building's depth wrong by 57%. Not slightly worse — unusable.

The mechanism is exactly what Part 13.3 described. The masker mistook the parallax of the tall building for motion and painted the building out, taking 45% of the keyframes down with it. Everything downstream then had less than half the evidence to work from.

This is why the pipeline's default is `mask_backend = "none"`, and it is a good example of the project's engineering being driven by measured results rather than by what sounds sophisticated.

CAREFUL HERE — RECONCILE THIS WITH THE README BEFORE SOMEONE ELSE DOES

The README states that a typical run reports "~0.5–3% error on known reference dimensions". Run B is consistent with that. **Run A, which is the run named `proj_demo`, is not** — and its name suggests it may be the one someone reaches for.

If you are demonstrating this, use the configuration from Run B: masking off, and supply the camera intrinsics if you have them. And if a judge finds the `proj_demo` numbers, the honest and impressive answer is the true one: "that run had optical-flow masking enabled, which mistakes building parallax for motion and cost us 45% of our keyframes. That is exactly why it is off by default, and it is documented in the troubleshooting guide." Knowing precisely why your own bad result was bad is a much stronger position than not having one.

24.4 The other numbers, and how to read them honestly

Two further figures from Run A are worth understanding, because both look alarming and only one is.

Surface accuracy of 7.3 m

The report's `surface_accuracy_m.median` for Run A is 7.31 m, which sounds catastrophic next to a 9-metre building. But this figure is computed uncropped, over every reconstructed point including those far outside the region the ground truth covers. The evaluation harness in Part 22 exists precisely to fix this — it registers with scale-locked ICP and crops to the ground-truth region first, and its own code comments note that the uncompensated version inflates the error several-fold. Read the **dimensional accuracy** table instead; that is the number that reflects what a user would actually do.

Processing-to-video ratio of 7.3

Run A took 14.6 seconds to process 2 seconds of video. Broken down: SfM 8.7 s, masking 4.1 s, everything else about 1.8 s. The masking alone cost more than a quarter of the run — for a result that made everything worse. With masking off it drops to roughly 5×. This is comfortably inside the brief's expectation that refined reconstruction "may take several times the video duration", though the sample is tiny and a real multi-minute flight would scale up considerably.

The provenance split

Run A's cloud was 55.5% high-confidence and 44.5% low-confidence, with nothing AI-assisted (densification was off). That split is itself useful information: nearly half the geometry was flagged as weakly observed, which is an honest reflection of a two-second single pass and exactly the sort of thing the confidence system exists to surface.

PART 25

Honest limits and how to demo this

What this system can and cannot do, stated plainly — and what to say about it.

25.1 What it genuinely does

- Turns a real drone video plus a GPS track into a real, triangulated 3D point cloud, using a reconstruction engine written from scratch that runs on any laptop with no graphics card, no COLMAP, and no downloaded models.
- Recovers real-world metres and a position on Earth from the GPS track, using a robust outlier-rejecting alignment.
- Measures distances, heights and areas — on the bundled ground-truth fixture, to within 0.6–3.5% under the right settings.
- Labels every point with how much it should be trusted, and enforces that labelling in the measurement code, the viewer and the exports.
- Reports honestly: it warns when it guessed the intrinsics, warns when it levelled the scene, warns when a measurement touches weak geometry, states that alignment residual is not accuracy, and prints its own limitations on every report.
- Probes and reports what is actually installed rather than claiming capabilities.
- Exports to seven formats and runs offline in Docker.

25.2 What it does not do

Limitation	The honest statement
No global bundle adjustment	The built-in engine re-triangulates points but never goes back and jointly refines the camera poses. Long sequences will drift. COLMAP is the answer and is supported as an option.
Sparse, not dense	Output is thousands of points, not millions. Dense multi-view stereo is not implemented; the depth prior is the substitute, and it is explicitly AI-assisted rather than measured.
MASt3R-SLAM and VGGT are stubs	The interface exists and is honest, but calling them raises "not implemented". Do not claim these run.
No texture mapping	The mesh carries per-vertex colours, not projected photographic texture. So "textured mesh" in the original brief is only partially delivered.
Semantic masking needs your own weights	Deliberately constructed with no weights so nothing downloads. Without a local checkpoint it produces nothing meaningful.
Optical-flow masking is unreliable on tall structures	Documented, off by default, and Part 24 quantifies the damage.
Absolute accuracy is GPS-limited	Ordinary GPS means metre-level absolute position. RTK/PPK is required for anything better, and the report labels which you had.
240 frames, 1280 px	Hard processing limits. Good for speed; they do cap the achievable detail.
Nothing recovers unseen surfaces	Not a limitation of the implementation — a limitation of physics. The far side of a building filmed from one side does not exist in the data.

25.3 Things to never say

CLAIMS THAT WOULD NOT SURVIVE SCRUTINY

- **"Centimetre accurate."** Not from consumer GPS, and not without measured ground truth to back it. The code's own rule: never claim centimetre accuracy without evidence.
- **"Our alignment RMSE is 0.98 m, so we are accurate to a metre."** This conflates consistency with accuracy, and the report explicitly warns against it. If someone else makes this mistake, correcting it is a good look.
- **"We use MASt3R-SLAM / VGGT."** The adapters are stubs.
- **"It reconstructs the whole scene."** It reconstructs what the camera saw.
- **"The mesh is measurable."** Measurements run on the point cloud. Poisson meshing spans holes.

25.4 What to say instead

The strongest framing is the one the project was built around. Most reconstruction software answers the question "what does this place look like in 3D?" This one answers a different and harder question: **"which parts of this 3D model am I allowed to trust?"**

A useful thirty-second version:

THE PITCH

"One drone pass gives you a video and a GPS track. We turn that into a 3D model you can actually measure on — real metres, real coordinates. The hard part isn't building the model; it's that a single pass physically cannot see everything, and most software quietly fills in the gaps so the result looks complete. We don't. Every point in our model is labelled with where it came from — properly observed, weakly observed, AI-inferred, moving object, or never seen — and measurements refuse to use anything but the geometry we actually measured. On our ground-truth test we're within about 3% on known dimensions, and we say exactly what our accuracy depends on rather than quoting a number we can't defend."

25.5 The five questions you will be asked

Question	The answer
"How do I know the AI didn't hallucinate that wall?"	Because AI-derived points are a separate layer with provenance code 2, coloured purple, and the measurement code removes them from the search before it finds the nearest point. You have to tick a box to include them, and if you do, the measurement carries a permanent warning. There is a test asserting this.
"How do you get real-world scale from one camera?"	We can't from the video alone — that's scale ambiguity, and it's a mathematical fact. We get it from the GPS track: fit the reconstructed camera path to the recorded positions with a RANSAC similarity transform, and the scale factor falls out. That's why the report labels its scale source as RTK, GPS, or relative.
"Why not just use COLMAP?"	We support it, and it's better for hard captures because it does global bundle adjustment. But it's a separate install that's awkward on Windows and wants CUDA for its best features. We wrote an engine that always runs, so the demo never fails on someone else's laptop, and COLMAP is a switch you flip when you have it.
"What's your accuracy?"	On our ground-truth fixture, 0.6% to 3.5% on known dimensions. Absolute position on Earth is GPS-limited — metre-level with consumer GPS, decimetre with RTK. And our alignment residual is not an accuracy figure; we report it separately and say so.
"What can't it do?"	Any answer from the table in 25.2, delivered without hedging. Knowing your own limits precisely is the most credible thing you can demonstrate — and it is the entire thesis of this product.

PART 26

Glossary

Every term used in this guide, in one line each. Terms are grouped by what they are about rather than alphabetically, because related ideas are easier to hold together.

26.1 Geometry and reconstruction

Baseline

The distance between two camera positions. Too small and depth is unstable; too large and images stop matching.

Bundle adjustment

Refining all camera poses and all 3D points together to minimise total reprojection error. The built-in engine here does not do it globally; COLMAP does.

Cheirality check

Verifying a triangulated point is *in front of* every camera that saw it. Points behind a camera are physically impossible and are discarded.

Dense reconstruction

Computing depth for every pixel, producing millions of points. Usually needs a GPU. Not implemented here.

Descriptor

A compact numerical summary of what a feature patch looks like, designed to stay similar under changes of angle and lighting.

DLT (Direct Linear Transform)

The linear-algebra method used to triangulate a point from many observations at once.

Epipolar constraint

The geometric relationship that valid matches between two images of the same scene must satisfy. Used to reject false matches.

Essential matrix

A compact encoding of how two cameras with known intrinsics are related. Used here for geometric verification and initialisation.

Feature / keypoint

A small distinctive image patch that can be recognised again from a different viewpoint.

Inlier / outlier

A data point that agrees with the fitted model / one that does not and is rejected.

Loop closure

Recognising you have returned to a previously seen place, allowing accumulated drift to be corrected.

Mesh

A continuous surface made of triangles, as opposed to loose points.

MVS (Multi-View Stereo)

The dense reconstruction stage that follows sparse SfM.

Normal

The direction a surface faces at a point. Needed for meshing and lighting.

Optical flow

An estimate of how far each pixel appears to have moved between two frames.

Parallax

The apparent shift of objects against their background as the viewpoint moves. Near things shift more. The basis of all depth from motion.

PnP (Perspective-n-Point)

Solving for a camera's pose given known 3D points and where they appear in its image.

Point cloud

A collection of 3D points, usually coloured, sitting on the surfaces the camera saw.

Poisson reconstruction

The standard method for turning an oriented point cloud into a smooth closed surface. Fills holes, which is both its virtue and its danger here.

Pose

A camera's position and orientation — six numbers.

RANSAC

Random Sample Consensus: fit a model to a small random sample repeatedly, keep the fit that the most data agrees with. The standard defence against outliers.

Registered

A frame successfully placed into the reconstruction with a solved pose.

Reprojection error

The pixel distance between where a 3D point should appear in an image and where its feature was actually detected. Under 1 px is good.

SfM (Structure from Motion)

Recovering both scene structure and camera motion from overlapping images, processed as a batch.

SIFT

Scale-Invariant Feature Transform. The classic feature detector and descriptor, used by this engine.

SLAM

Simultaneous Localisation and Mapping. The same problem as SfM, arranged to run live.

Sparse cloud

One point per triangulated feature. Accurate but skeletal. What this system produces.

SVD (Singular Value Decomposition)

A matrix operation that appears throughout: triangulation, the Umeyama fit, plane fitting for area measurement.

Track

All the sightings of one physical point across many frames, grouped together.

Triangulation

Computing a 3D position by intersecting rays from two or more known camera positions.

Triangulation angle

The angle at which those rays meet. Wider is better determined. A key input to the confidence score.

Voxel

A 3D pixel — a small cube of space. Used for downsampling to even out point density.

26.2 Cameras and imaging

Calibration

Measuring a specific camera's intrinsics and distortion, usually by photographing a chessboard from many angles.

Clipping

Pixels pushed to pure black or pure white, where all texture information is permanently lost.

Codec

The compression scheme a video is stored in, such as H.264.

Distortion coefficients

Numbers (k_1 , k_2 , k_3 , p_1 , p_2) describing how the lens bends straight lines.

f_x , f_y

Focal length in pixels, horizontally and vertically. Controls how "zoomed in" the camera is.

c_x , c_y

The principal point — where the lens's optical centre falls on the sensor, in pixels.

Frame / keyframe

One still image from the video / one that was selected for use.

Intrinsics

The camera's internal parameters: f_x , f_y , c_x , c_y , and distortion.

Laplacian variance

A single-number sharpness score. Low means blurry.

Monocular

Using one camera. Implies scale ambiguity.

Nadir

Pointing straight down. A nadir flight is one where the camera looks vertically downward.

Ray

The line out from a camera through one pixel. A single image gives direction but not distance.

26.3 Position and coordinates

ATE (Absolute Trajectory Error)

How far estimated camera positions are from true ones, after a Sim(3) alignment.

ECEF

Earth-Centred Earth-Fixed. Metres from the planet's centre. Used as a conversion bridge.

Ellipsoid vs geoid

The smooth mathematical Earth shape vs the actual lumpy sea-level surface. GPS altitude is above the ellipsoid.

ENU

East, North, Up. Plain metres from a chosen local origin. Where all the real work happens.

EPSG code

A universal number identifying a coordinate system. 4979 is WGS84 with height; 4978 is ECEF.

GCP (Ground Control Point)

A surveyed point of precisely known position, used to scale or verify a model.

Georeferenced

Positioned on Earth, so every point has a real latitude and longitude.

GNSS / GPS

Satellite positioning. Consumer accuracy is roughly 2–10 m horizontally, worse vertically.

IMU

Inertial Measurement Unit — measures acceleration and rotation.

Metric scale

Having correct real-world size in metres, not just correct shape.

PPK

Post-Processed Kinematic. Like RTK but corrections applied after the flight.

RMSE

Root Mean Square Error. A typical error that penalises large errors more heavily.

RTK

Real-Time Kinematic. Centimetre-accurate GPS using a fixed base station broadcasting live corrections.

Scale ambiguity

The fact that a video cannot reveal the size of what it shows.

SE(3) / Sim(3)

Rotation and translation / rotation, translation and uniform scale. Sim(3) is what aligns the model to GPS.

Umeyama's algorithm

The direct method for computing the best-fitting similarity transform between two point sets.

WGS84

The standard model of Earth's shape used by GPS. Latitude, longitude, altitude.

Yaw / pitch / roll

Compass direction / nose up-down / tilt sideways.

26.4 This project's own terms

AI_ASSISTED

Provenance class 2, purple. From an AI depth estimate. Excluded from measurement unless opted in.

Alignment residual

How well the reconstructed camera path agrees with the GPS track. **Not** an accuracy figure.

Completeness

What fraction of the true surface was reconstructed within a tolerance.

Depth prior

A monocular AI depth estimate, anchored to real SfM points to give it metric scale.

DYNAMIC_EXCLUDED

Provenance class 3, red. Belonged to something moving.

Gravity / level correction

Rotating the model so the dominant ground plane is horizontal.

OBSERVED_HIGH_CONFIDENCE

Provenance class 0, green. Confidence ≥ 0.6 . The only class measurable by default.

OBSERVED_LOW_CONFIDENCE

Provenance class 1, amber. Genuinely seen but weakly. Measurable with a warning.

Provenance

The trust class attached to every point. The core idea of the whole product.

Relative scale mode

What happens with no usable GPS: correct shape, no size, no map position.

Scale source

Whether size came from RTK, ordinary GPS, or nothing (relative).

Splat mode

Drawing points as soft overlapping blobs so a sparse cloud reads as a surface.

Trust map

The provenance colouring in the viewer.

UNOBSERVED

Provenance class 4, grey. No camera ever saw it. The honest hole.

26.5 Software terms

Adapter

A wrapper letting an external tool plug into a fixed interface, so it can be swapped or absent.

Air-gapped

Physically disconnected from any network.

API / endpoint

The set of requests one program can make to another / one specific such request.

Docker

Packaging an application with all its dependencies so it runs identically anywhere.

FastAPI / Pydantic

The Python web framework used here / the validation library it is built on.

Monorepo

One repository containing several components developed together.

Open3D

The 3D-processing library used for downsampling, normals, plane fitting and meshing. Optional throughout.

OpenCV

The image-processing library the whole reconstruction engine is built on.

React / TypeScript / Vite

The interface library / typed JavaScript / the build tool.

REST

The convention of using GET, POST and DELETE on resource paths.

SHA-256

A cryptographic fingerprint of a file. Any change produces a completely different one.

SQLite / SQLAlchemy

A whole database in a single file / the library for using databases as Python objects.

SSE (Server-Sent Events)

A held-open connection over which a server pushes updates to a browser.

Three.js

The browser 3D graphics library used for the point-cloud viewer.

Union-find

An algorithm for grouping connected things. Used here to build feature tracks.

Unit / integration test

A test of one function in isolation / a test of the whole system end to end.

PART 27

The one-page cheat sheet

Everything compressed. If you read only this page, you will still be able to hold a conversation about the project.

IN ONE SENTENCE

Drishti3D takes one drone video plus a GPS track and produces a georeferenced, metrically scaled 3D point cloud you can measure on, where **every point is labelled with how much it deserves to be trusted** — and measurements refuse to use the parts that were inferred rather than observed.

The two facts everything follows from

1. **Video alone cannot tell you size.** A big thing far away and a small thing nearby look identical. GPS breaks the tie.
2. **You cannot measure what was never filmed.** A single pass never sees the far side. AI can guess; a guess is not a measurement.

The five trust classes

● **0 Observed high** — measurable · ● **1 Observed low** — measurable, flagged · ● **2 AI-assisted** — opt-in only · ● **3 Dynamic** — excluded · ● **4 Unobserved** — a hole, honestly

Split at confidence 0.6. Confidence = $0.40 \times \text{observations} + 0.35 \times \text{low reprojection error} + 0.25 \times \text{wide triangulation angle}$.

The fourteen stages

ingestion → telemetry → frames → quality → sync → keyframes → masking → **sfm** → densify → **georegistration** → fusion → mesh → report → exports

How it works, in six steps

1. Find distinctive patches (SIFT) in each keyframe.
2. Match them across frames; reject bad matches with the ratio test and RANSAC on the essential matrix.
3. Group matches into tracks with union-find; triangulate each track by DLT; reject anything behind a camera, over 4 px reprojection error, or under 1° triangulation angle.
4. Add remaining cameras one at a time by PnP + RANSAC; re-triangulate every fourth camera.
5. Fit the camera path to the GPS track with a RANSAC Sim(3) (Umeyama) — this is where real metres arrive. Then level the ground plane.

6. Clean, label, mesh, report, export.

The numbers that matter

Registered keyframes	The single best health indicator. Near 100% is good; 55% means trouble.
Median reprojection error	Under 1 px good; over 2–3 px means bad intrinsics or bad matches.
Mean track length	Higher is better. Near 2 is fragile.
Alignment RMSE	Consistency, not accuracy. Never quote as accuracy.
Dimensional error vs ground truth	The only real accuracy figure. 0.6–3.5% on the bundled fixture with correct settings.
Scale source	<code>rtk / gps / relative</code> — tells you which accuracy regime you are in.

Settings that matter

- **Masking: leave it off.** Optical-flow masking mistakes building parallax for motion. It cost 45% of keyframes and 57% dimensional error in the bundled sample.
- **Supply intrinsics if you have them.** Otherwise the focal length is guessed at $0.9 \times$ the larger dimension and the run carries a warning.
- **Preset:** balanced by default; quality gives four times the keyframes and much longer runtime.
- **Densify: only if you want visual completeness**, and know everything it adds is purple and un-measurable.

Running it

```

pip install -e reconstruction
pip install open3d laspy -r backend/requirements.txt
python sample_data/generate_sample.py sample_data/synthetic
cd backend && uvicorn app.main:app          # terminal 1  -> :8000, docs at /docs
cd frontend && npm install && npm run dev   # terminal 2  -> :5173
pytest tests/ -q                          # the tests
python -m eval.run_eval --self-test       # accuracy self-test, no downloads
docker compose up --build                 # or the whole thing at once

```

Never claim

Centimetre accuracy without ground truth · that alignment residual is accuracy · that MAST3R-SLAM or VGGT run (they are stubs) · that the mesh is measurable · that unseen surfaces were reconstructed.

THE LAST WORD

The impressive thing about this project is not that it builds a 3D model — plenty of software does that. It is that it knows, and says, which parts of its own answer are worth believing. That is a harder problem, a rarer one to solve, and the one that actually matters when someone makes a decision based on the number it gives them.